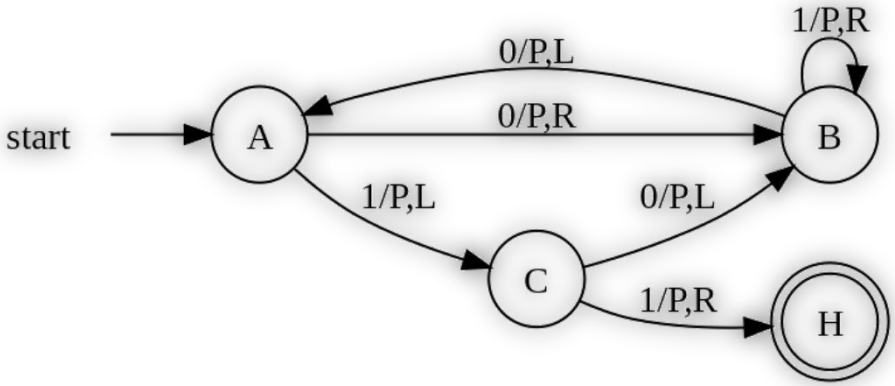
Digital Logic Design and Application

Topic 2 - Principles for Digital design

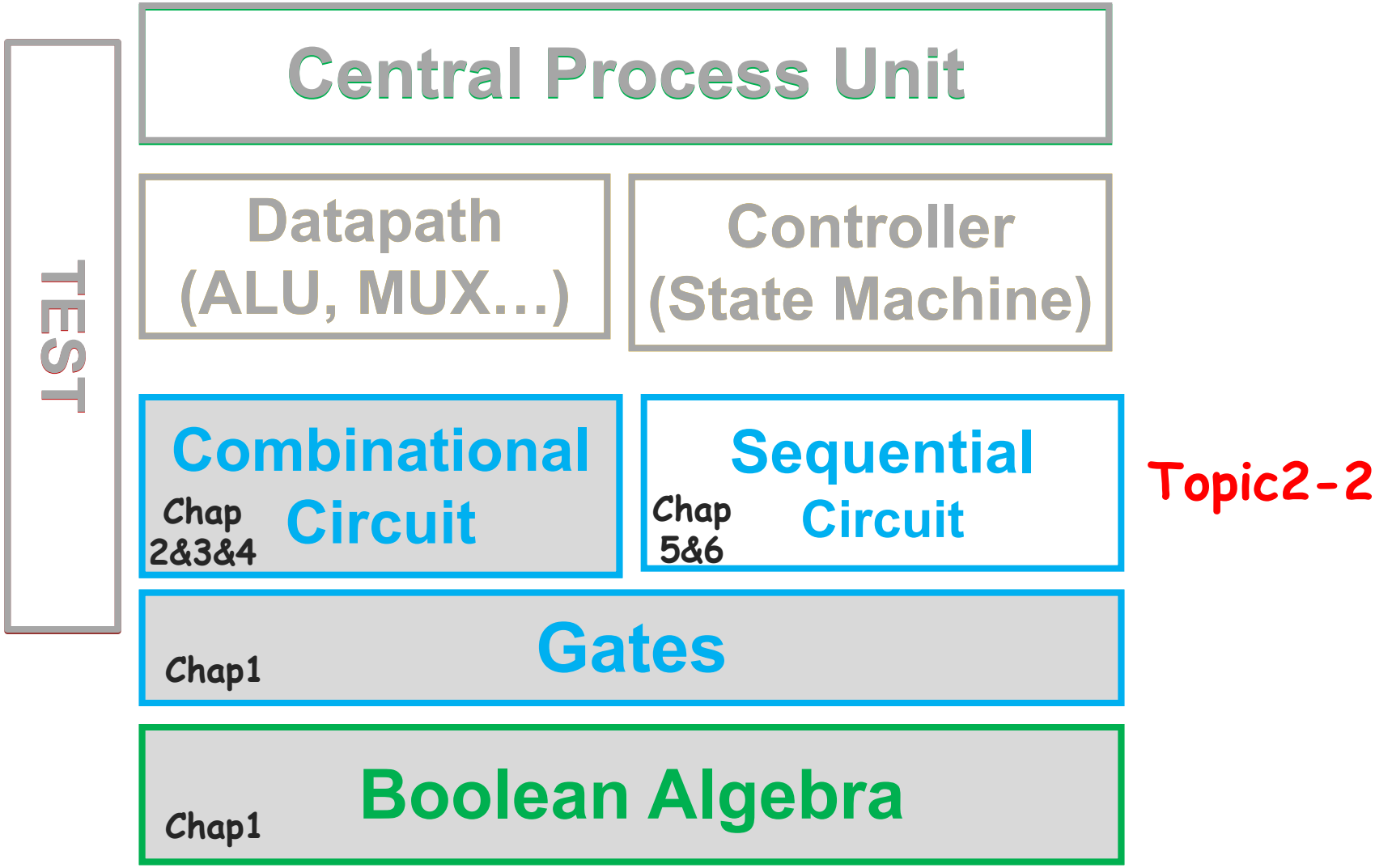
Yan Bo

School of Information & Communication Engineering, UESTC

Email: yanboyu@uestc.edu.cn



Where we are


TEST
Central Process Unit
Datapath
(ALU, MUX…)
Controller
(State Machine)
Combinational
Circuit
Chap
2&3&4
Sequential
Circuit
Chap
5&6
Topic2-2
Gates
Chap1
Boolean Algebra
Chap1

Topic2： Principles for digital design

Combinational Logic

- Gates and Logic
- Function Description
- Design & Analysis

Sequential Logic

- DFF/Latch, Register & Memory
- Mealy/Moore State machine
- Design & Analysis

Testbench

- Test module
- Stimulus Generator
- Response Monitor

Topic2： Principles for digital design

Combinational Logic

- Gates and Logic
- Function Description
- Design & Analysis

Sequential Logic

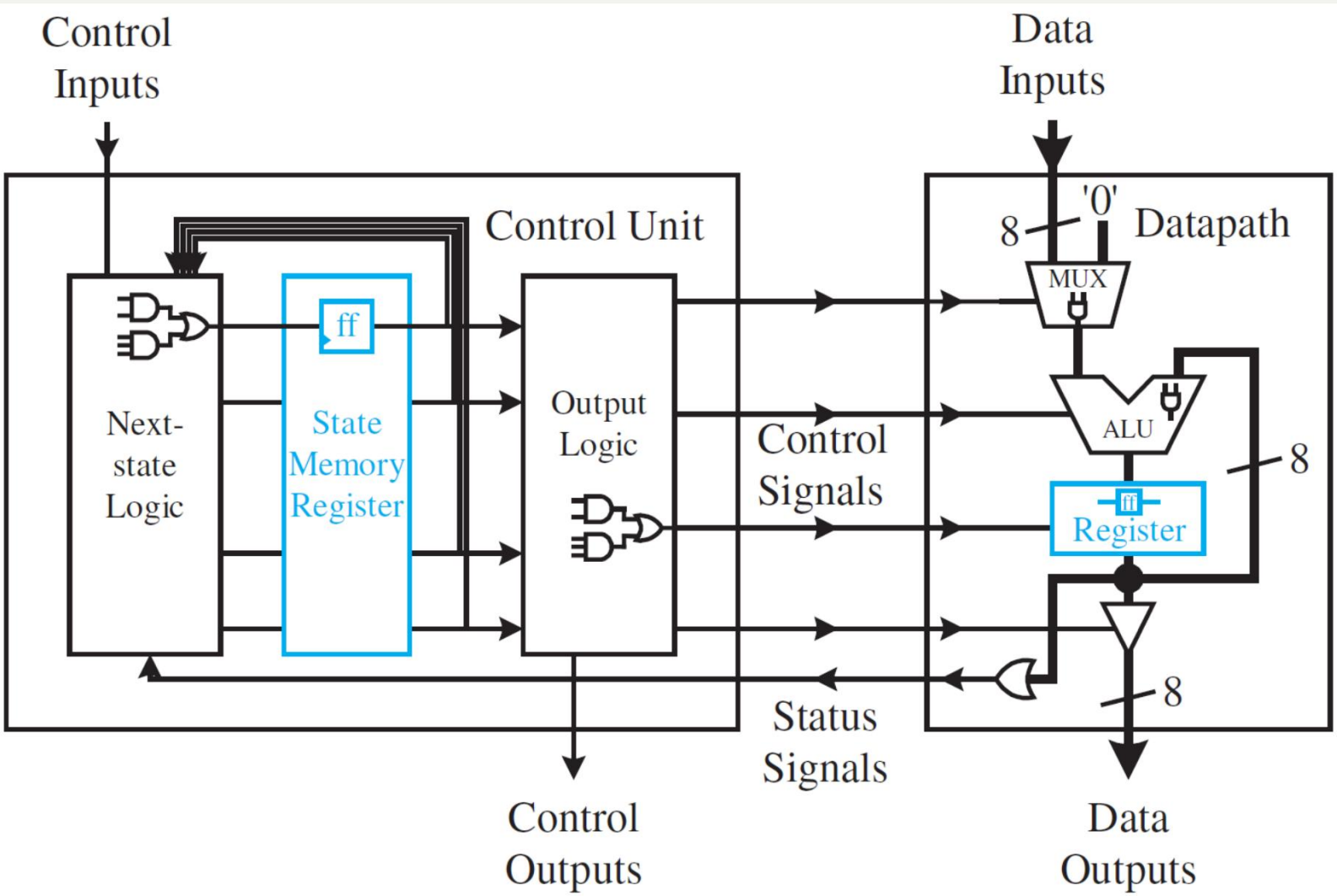
- **DFF/Latch, Register & Memory**
- Mealy/Moore State machine
- Design & Analysis

Testbench

- Test module
- Stimulus Generator
- Response Monitor

Organization of Center Processor Unit

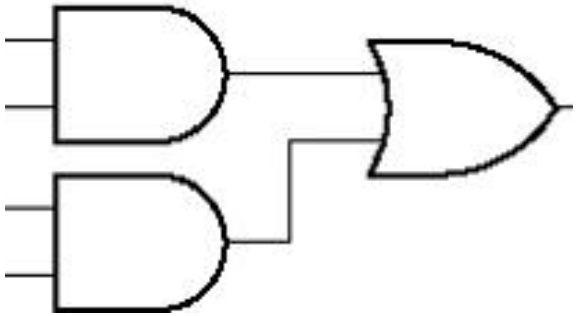
P157



What is Sequential Circuit

We need some devices which can remember/ storage the current state. And the controll circuit is a Sequen Logic, which outpu pends on current as w past inputs

Flip-Flop, Register, Memory(RAM,ROM), etc.



The output of combinational logic circuit only depends on current inputs because it has no storage devices.

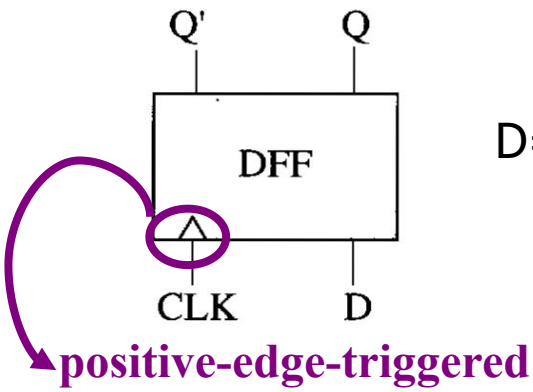
Combinational vs. Sequential

- Combinational Circuit
  - No storage devices
    - ⇒ The output only depends on the current inputs
  - The most basic component
    - ⇒ gates: NAND, NOR

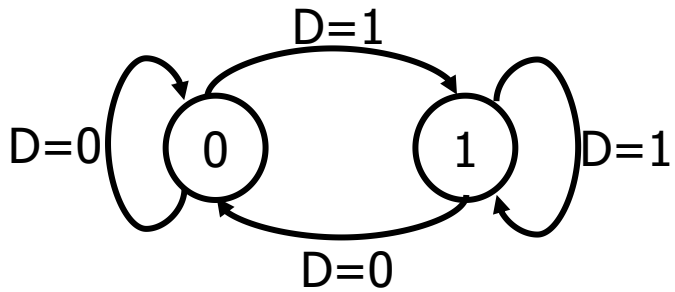
- Sequential Circuit
  - Combinational circuit + storage devices
    - ⇒ The output depends on the current as well as past inputs(STATE)
  - The most basic component
    - ⇒ Flip/Flop: DFF

D-type Flip-Flops

Logic Symbol



State Transiton Graph



Verilog code

Chap5.11

```
module DFF1 (
    input    CLK,D,
    output  reg  Q          );
  always @(  posedge CLK   )
    begin
        Q <= D    ;
    end
endmodule
```

Feature Function

$Q^+ = D$

Function table

D	CLK	Q	Q'
0	↑	0	1
1	↑	1	0
X	↓	Hold	
X	↓	Hold	
X	0	Hold	
X	1	Hold	

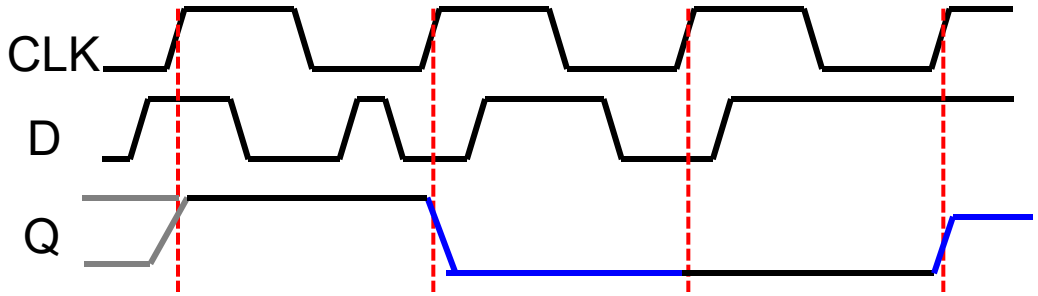
State transition table

Bistable circuit

D	Q	Q^+
0	X	0
1	X	1

State

Timing Diagram



F/F Changes its outputs/state only at the Rising/ Positive or Falling/Negative edge of a controlling CLK signal.

D Flip-Flop with other inputs

Chap5.10

```
module Flop (reset, clk, din, qout );
    input  reset, clk, din;
    output qout;
    reg    qout;

    always @(negedge clk)
      begin
        if (reset)  qout <= 1'b0;
        else        qout <= din;
      end

    always@( negedge clk or posedge reset)
      begin
        if (reset)  qout <= 1'b0;
        else        qout <= din;
      end

endmodule
```

Flip-Flop triggers on the falling edge of *clk* Input

Synchronous *reset*

Asynchronous *reset input*

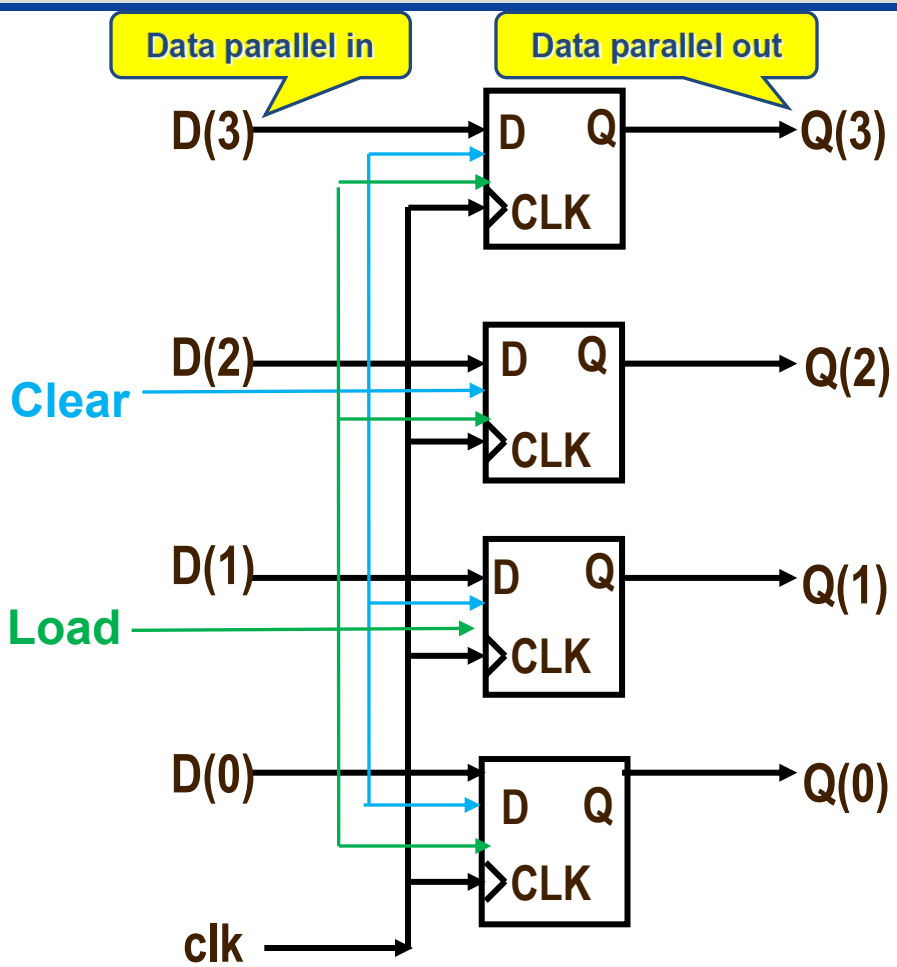
Non-blocking Assignments for registers

```
always @( * )
  begin
    if ( sel )  q = a;
    else        q = c;
  end
```

Blocking Assignments for combinational circuit

Assign Statements: `assign sum = a ^ b ^ cin;`

Register



Chap5.12

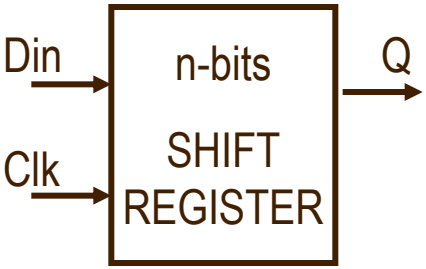
```
module register
    #(parameter n = 4)
( input              CLK, Clear, Load,
  input      [n-1:0] D,
  output reg [n-1:0] Q
);
  always @ ( posedge CLK or posedge Clear)
  begin
      if (Clear == 1)
          Q <= {n  {1'b0} };
      else   if (Load)
          Q <= D;
  end
endmodule
```

In typically , a register is a kind of device for a collection of flip-flops used to hold a bus (parallel)

A 4-bit Register with asynchronous Clear & synchronous Load

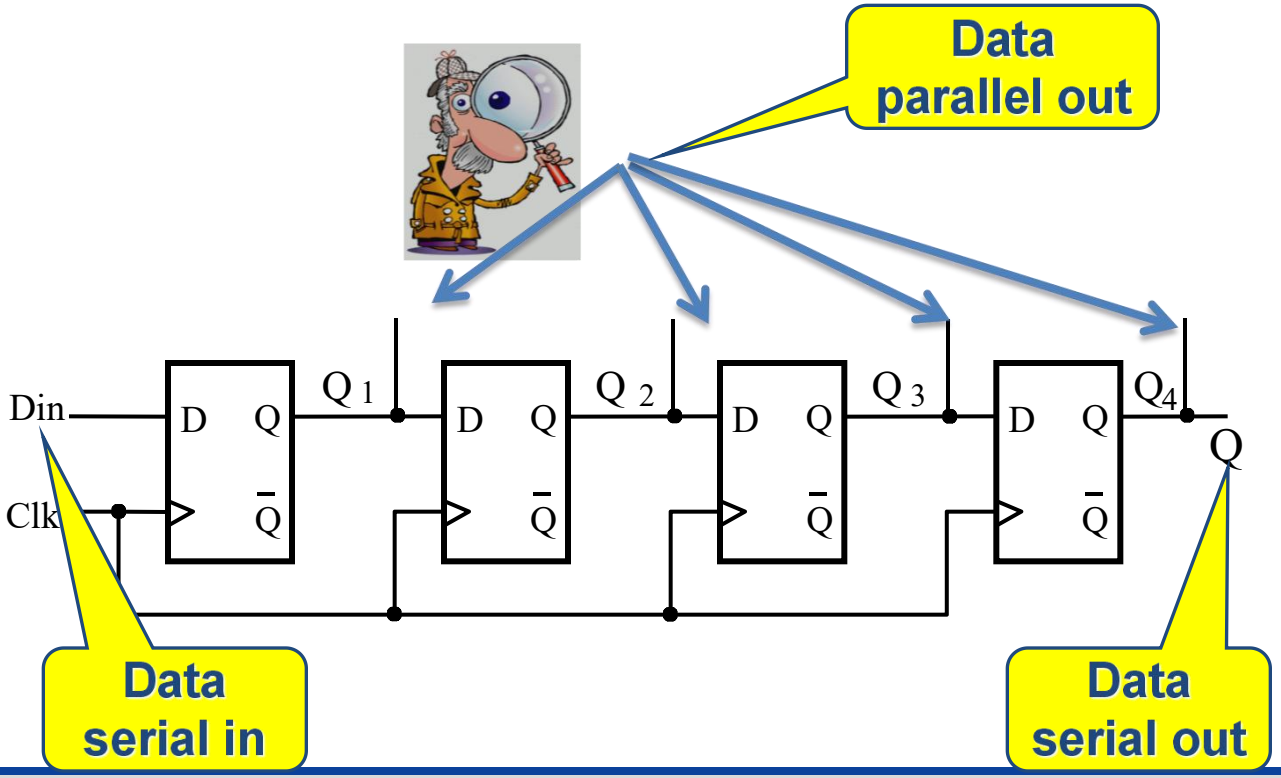
Shift Register

Chap5.15



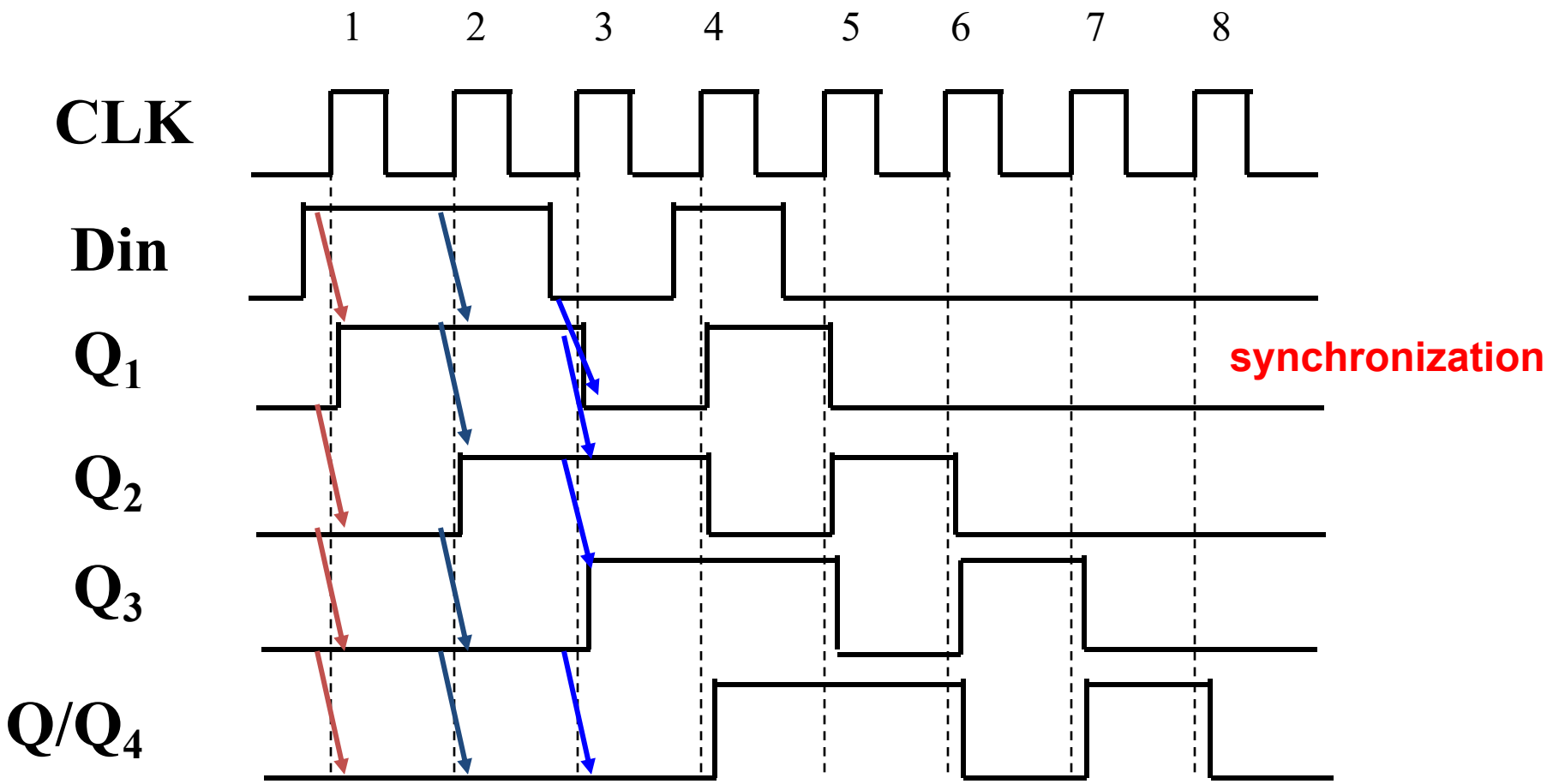
A 4-bits Shift Register

- provides 4 clocks Delay, can be used to delay a signal by 4 clock ticks
- remembers 4 previous input



	Din	Q_1	Q_2	Q_3	$Q = Q_4$
t_0	1				
t_1	0	1			
t_2	1	0	1		
t_3	1	1	0	1	
t_4	1	1	1	0	1
t_5	0	1	1	1	0
t_6	0	0	1	1	1
t_7	0	0	0	1	1

Shift Register waveform



Verilog code for 4-bit shift Register

```
module shiftreg (A, E, clk, rst);
    output  A;
    input   E, clk, rst;
    reg     A, B, C, D;
    always @ (posedge clk or posedge rst)
      begin
        if (rst)
          begin
            A <= 0; B <= 0; C <= 0; D <= 0;
          end
        else
          begin
            A <= B;  B <= C;  C <= D;  D <= E;
          end
      end
endmodule
```

Asynchronous Set/Reset

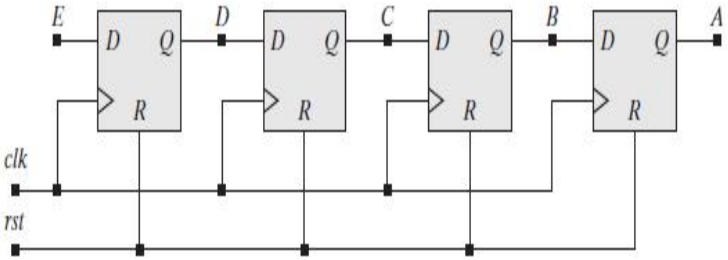
Synchronous Set/Reset

Positive Edge-Trigger

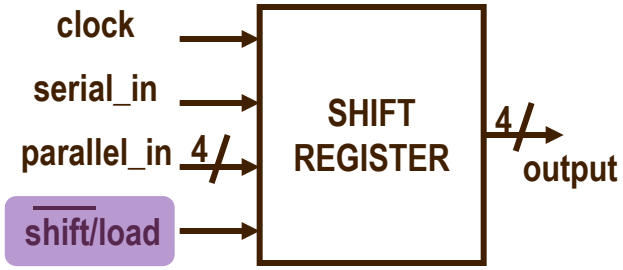
Sensitivity List

Non-blocking Assignment

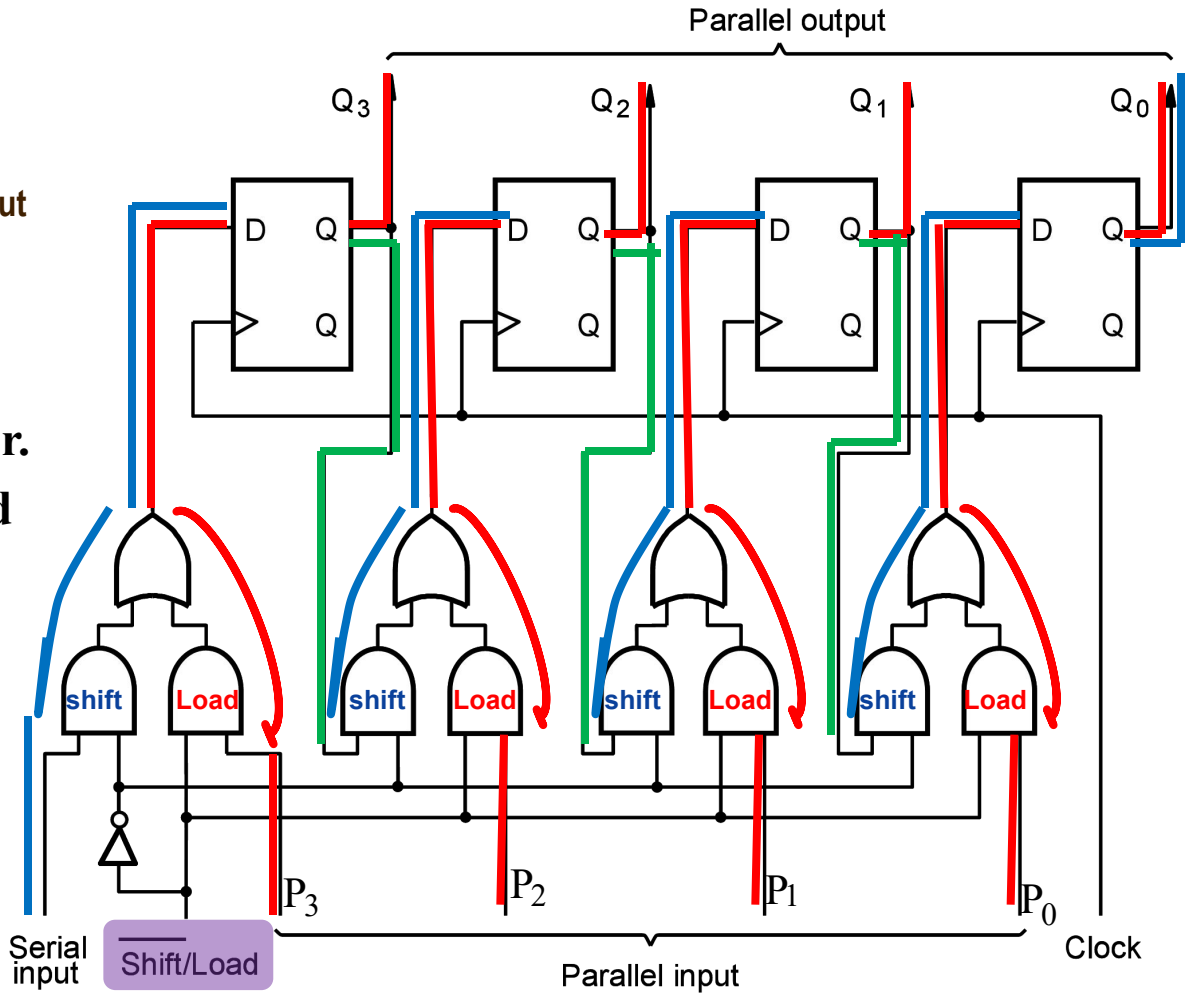
```
always @ (posedge clk)
  begin
    if (rst)
    .......
  end
```



Parallel Access Shift Register



- **Parallel in-Parallel out** can be used as a normal storage register.
- **Serial in- Serial out** can be used as a normal shift register with 4 clocks delay.
- **Serial in-Parallel out** can be used to perform Serial-to-Parallel conversion.
- **Parallel in- Serial out** can be used to perform Parallel-to-Serial conversion.



Verilog code for 8-bit Universal Shift Register

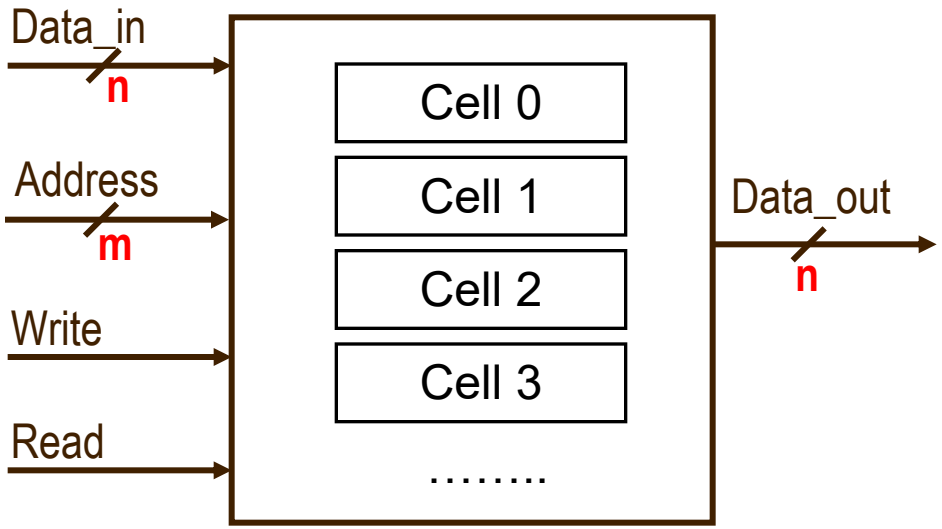
```
module ShiftRegister8
   (input sl, sr, clk, input [7:0] ParIn,
    input [1:0] m, output reg [7:0] ParOut);

always @(negedge clk)
  begin
      case (m)
          0: ParOut <= ParOut;                    //Does Nothing
          1: ParOut <= {ParOut [6:0], sl};        //Shifts Left
          2: ParOut <= {sr, ParOut [7:1]};        //Shifts Right
          3: ParOut <= ParIn;                     //Loads
          default: ParOut <= 8'bX;                //Undefine
      endcase
   end
endmodule
```

Random Access Memory (RAM)

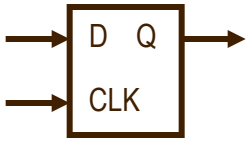
Chap5.14

- Addressable
- Can be read and written
- More efficient and cheaper than registers for storing large amounts of data

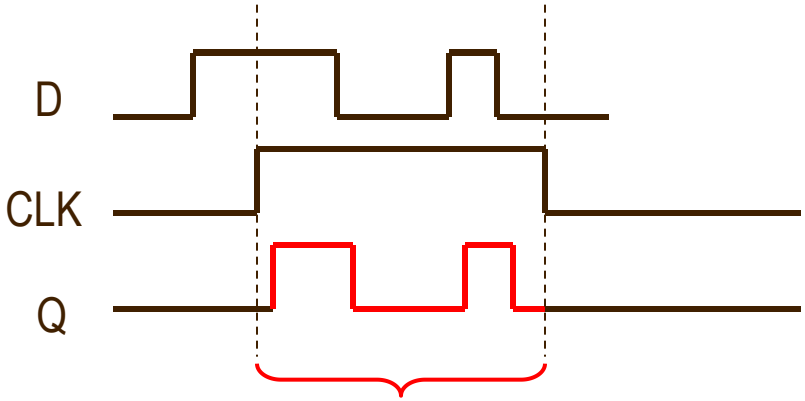


D Latch vs. D Flip-Flop

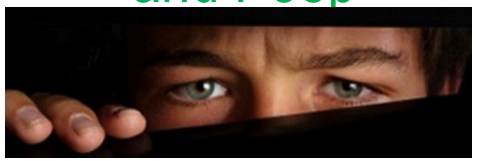
Chap5.5



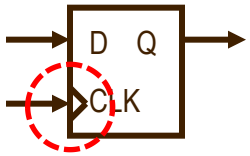
D Latch



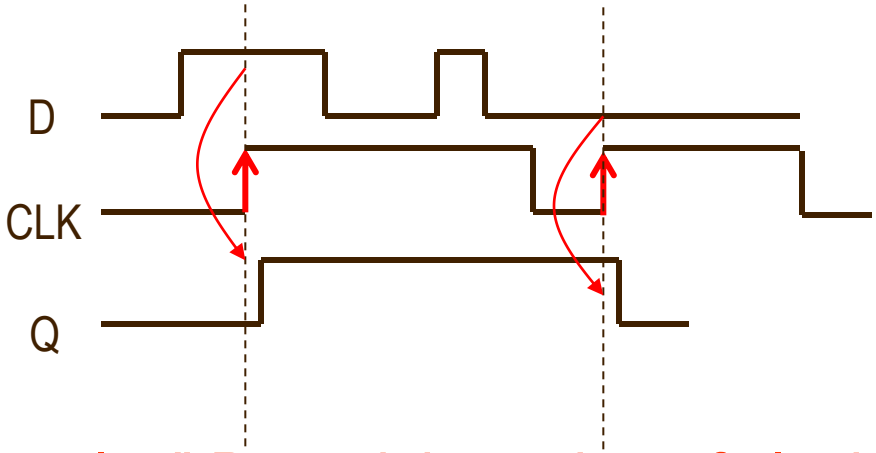
Open a Window and Peep



Latch transparent when clock is high



D Flip-Flop



Snapshot Camera

"Samples" D on rising edge of clock

Topic2： Principles for digital design

Combinational Logic

- Gates and Logic
- Function Description
- Design & Analysis

Sequential Logic

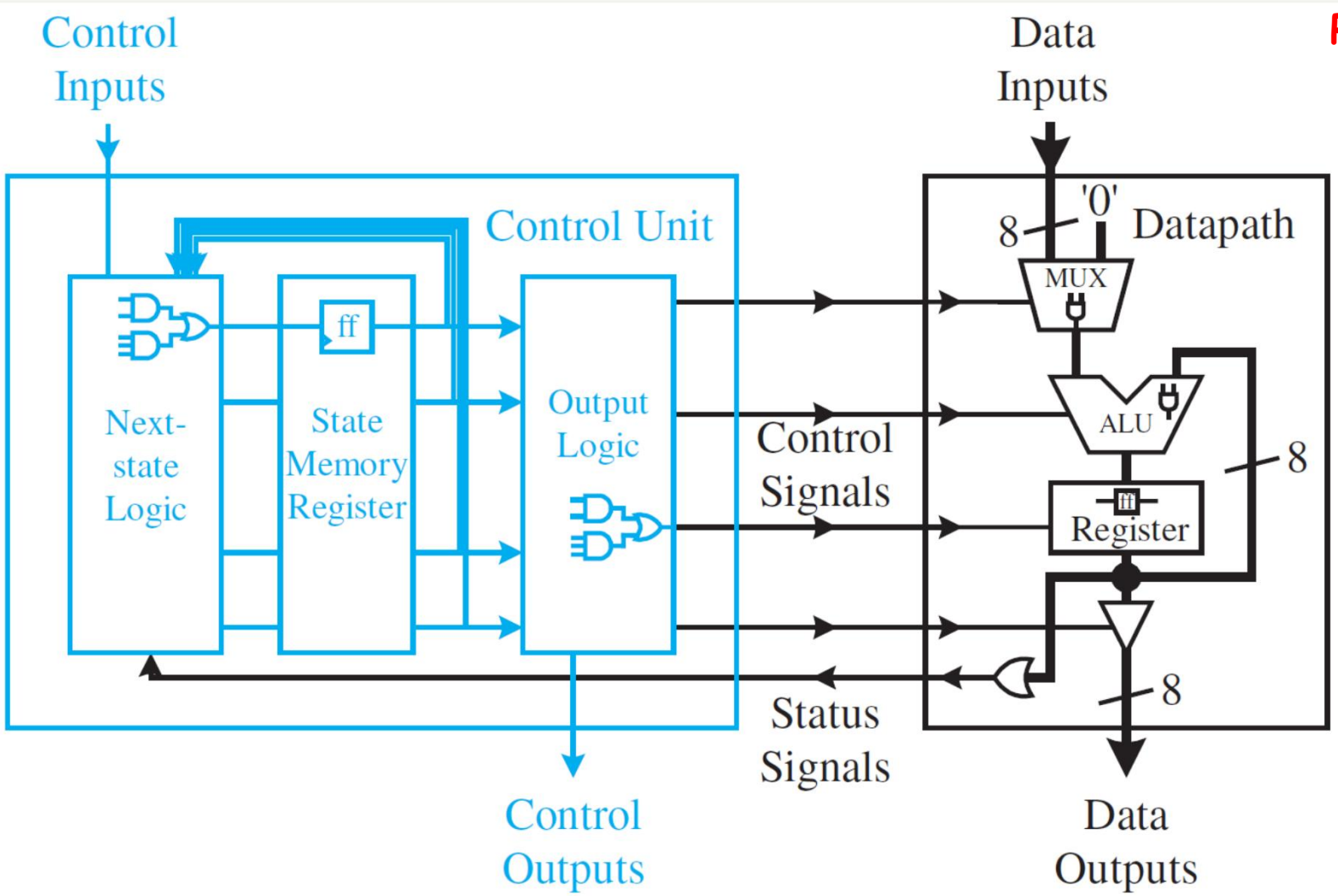
- DFF/Latch, Register & Memory
- **Mealy/Moore State machine**
- Design & Analysis

Testbench

- Test module
- Stimulus Generator
- Response Monitor

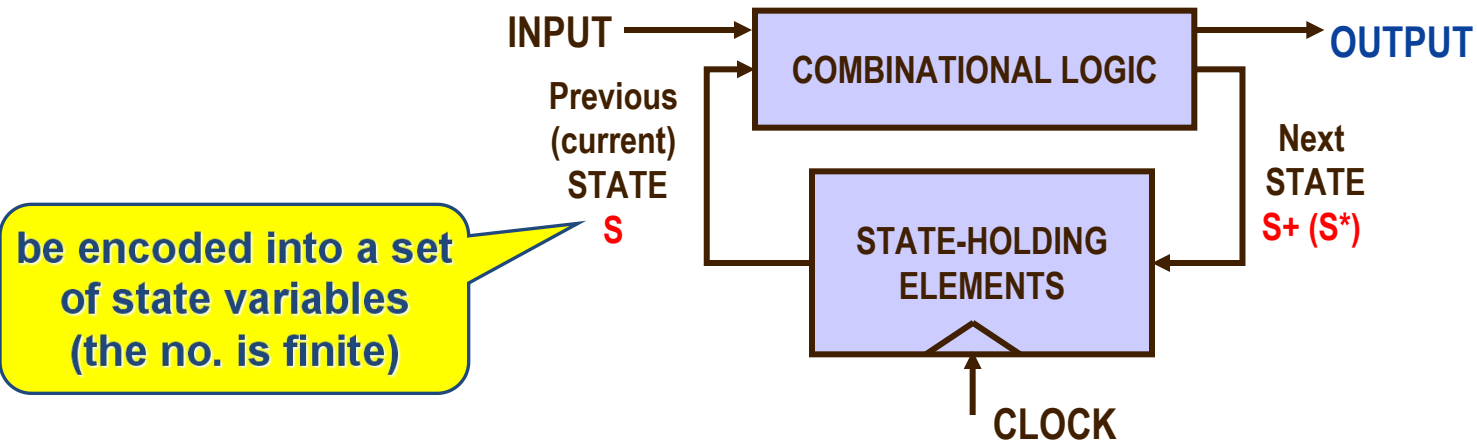
Organization of Center Processor Unit

P215



Sequential Logic Circuit

- **Sequential circuit consists of**
  - Combinational circuit
  - Feedback circuit to hold & feed the previous(current) information (i.e., state)
  - The 'clock' signal to keep circuits in lock step (synchronous, meaning that events always occur on the edge of a clock)



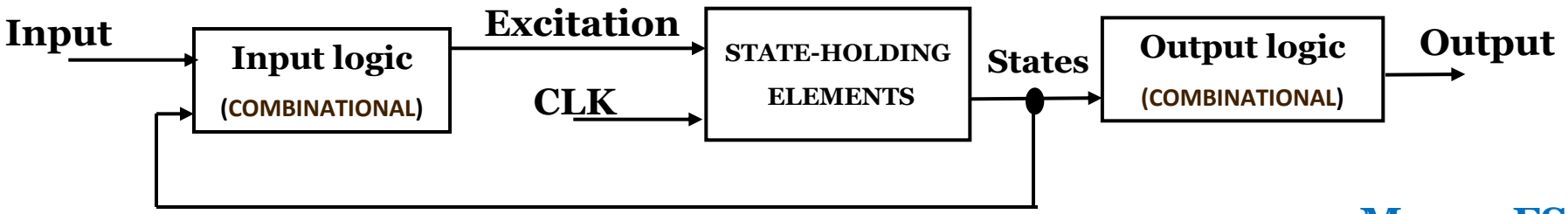
Finite State Machine (FSM)

- **There are two basic structures of output types**
  - **Mealy FSM** the outputs depend on the Previous(Current) State and the Inputs
  - **Moore FSM** the outputs depend on the Previous(Current) State only

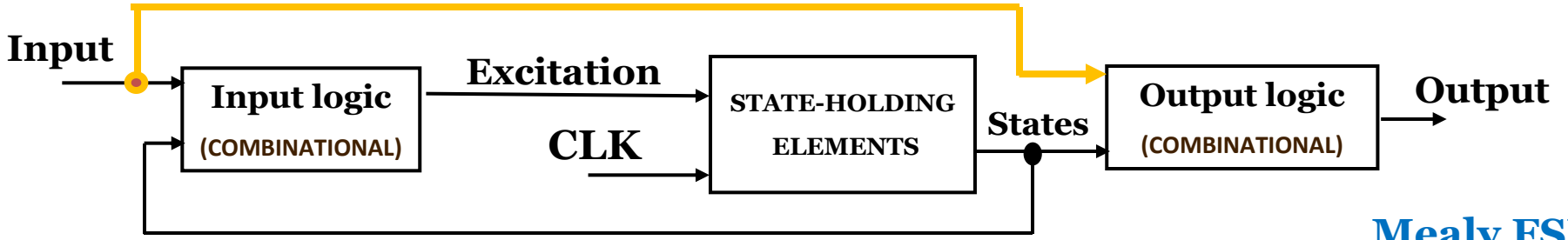
Digital Logic Circuits

Chap6.1

Combinational Logic



Moore FSM



Mealy FSM

Moore vs. Mearly

State transition/Output table

Input

Output

S	EN 0	EN 1	MAXS
A	A	B	0
B	B	C	0
C	C	D	0
D	D	A	1
	S*		

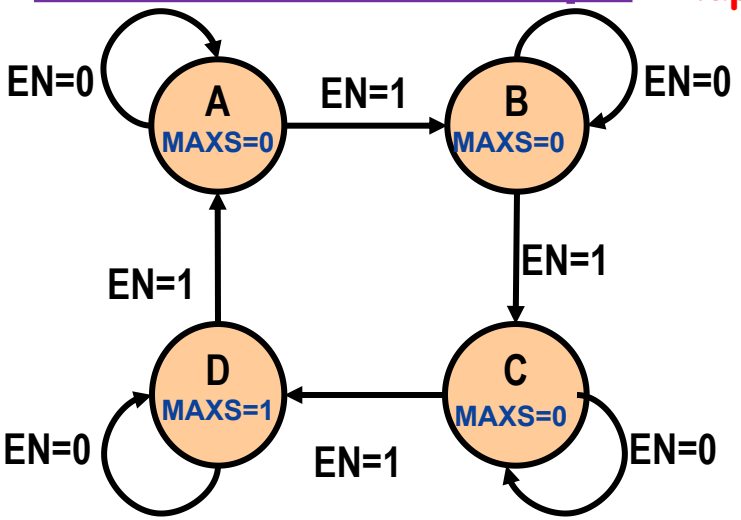
Current state

Next state

Moore

State Transition Graph

Chap6.2



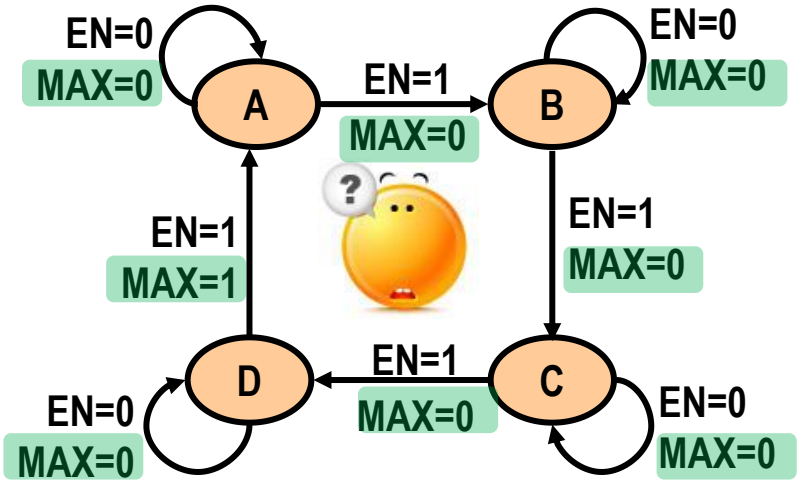
Input

S	EN 0	EN 1
A	A, 0	B, 0
B	B, 0	C, 0
C	C, 0	D, 0
D	D, 0	A, 1
	S*, MAX	

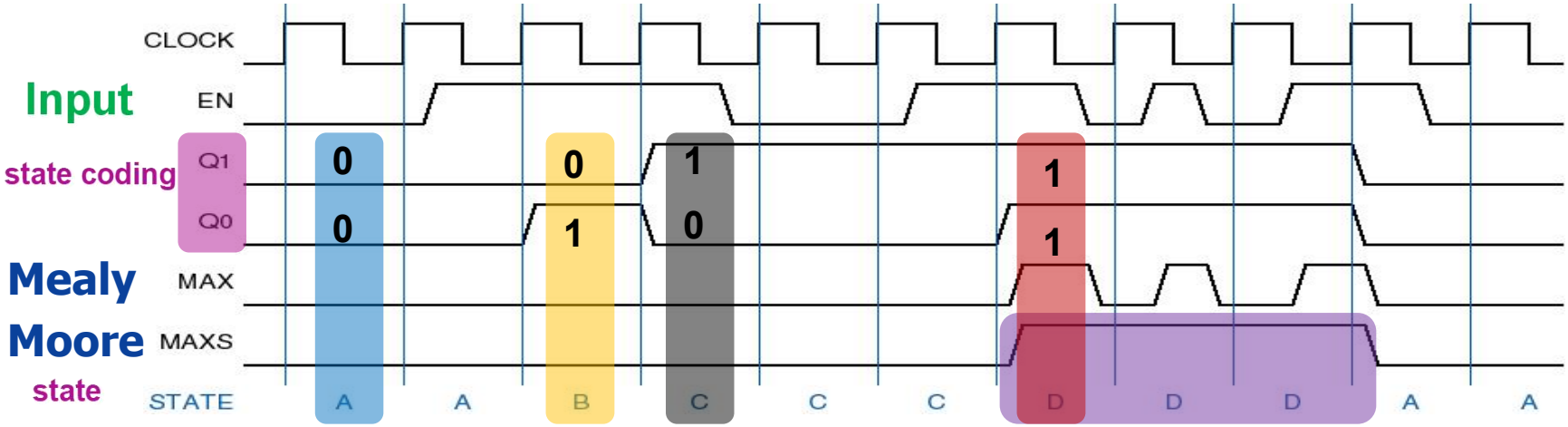
Current state

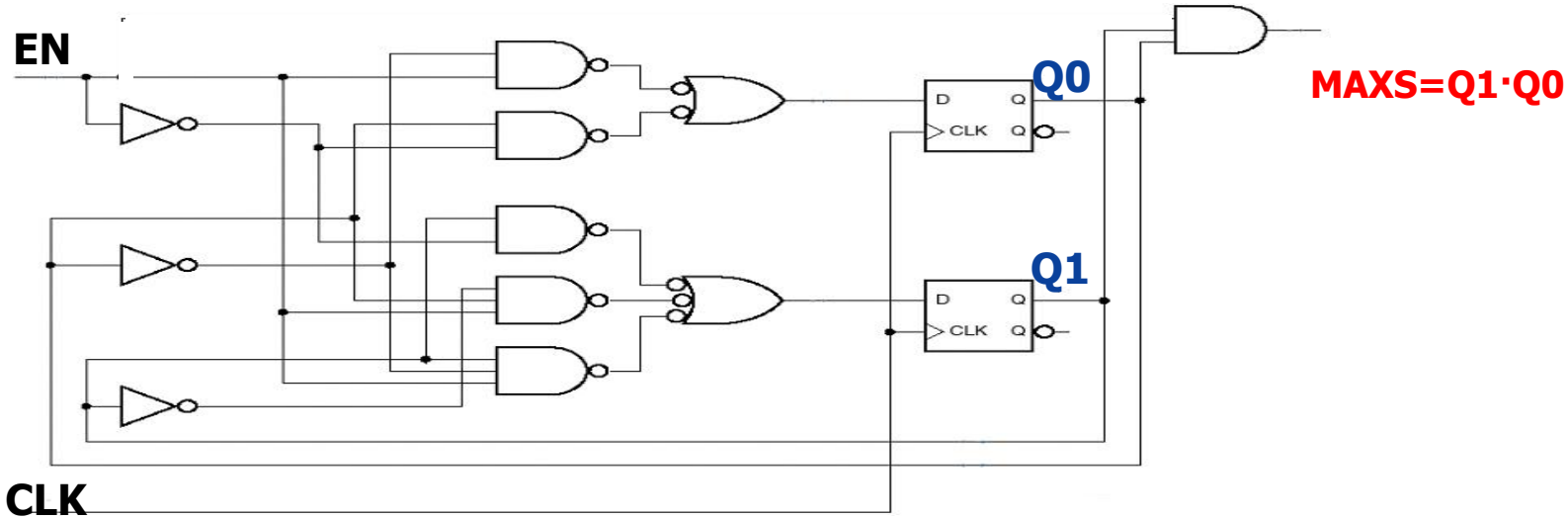
Next state/ Output

Mealy



Moore vs. Mearly





Topic2： Principles for digital design

Combinational Logic

- Gates and Logic
- Function Description
- Design & Analysis

Sequential Logic

- DFF/Latch, Register & Memory
- Mealy/Moore State machine
- **Design & Analysis**

Testbench

- Test module
- Stimulus Generator
- Response Monitor

About our course: Logic circuit(system)

- Combinational Circuit — Design & Analysis
  - No storage devices
    - ⇒ The output only depends on the current inputs
  - The most basic component
    - ⇒ gates: NAND, NOR
  - The most commonly used modules
    - ⇒ Decoder/Encoder, MUX/DeMUX, ALU, and so on
    - ⇒ Parity Check circuit, commonly used in communication system
- Sequential Circuit — Design & Analysis
  - Combinational circuit + storage devices
    - ⇒ The output depends on the current as well as past inputs(STATE)
      - ⇒ Mealy state machine: the output depends on current input and STATE
      - ⇒ Moore state machine: the output only depends on the STATE
  - The most basic component
    - ⇒ Flip/Flop: DFF
  - The most commonly used modules
    - ⇒ Register, Counter, and so on
    - ⇒ Serial Signal Detector, Serial Signal Generator

Design & Analysis of Sequential Circuits

- **Digital Circuit Design is:**
  - To descript the function
    - ⇒ truth table ,**State Transition/Output Table**
    - ⇒ K map ,**State Transition Graph**
    - ⇒ logical algebra expressions/equations **Excitation equation** **Output equation** **State transition equation**
    - ⇒ circuit diagram (Schematic at Module level, RTL level, etc.)
    - ⇒ timing diagram (Waveform)
    - ⇒ Hardware Description Language code (VHDL, Verilog, System C, etc.)
  - To simplify and realize the hardware
    - ⇒ gates or other devices , **Flip-Flop**
    - ⇒ Programmable Logic Devices
    - ⇒ Application Special Integrate Circuit
- **Digital Circuit Analysis is:**
  - To determine the behavior of the circuit
  - To verify the correctness of the circuit
  - To assist in converting the circuit to a different form

Design & Analysis of Sequential Circuits

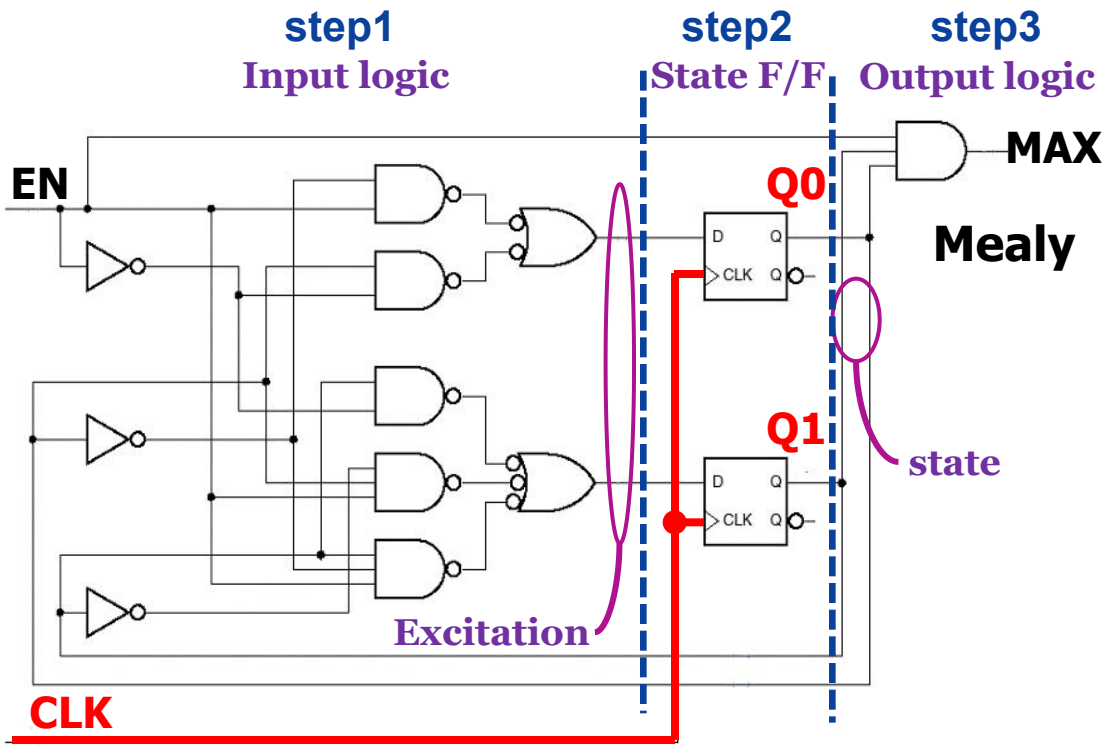
P219

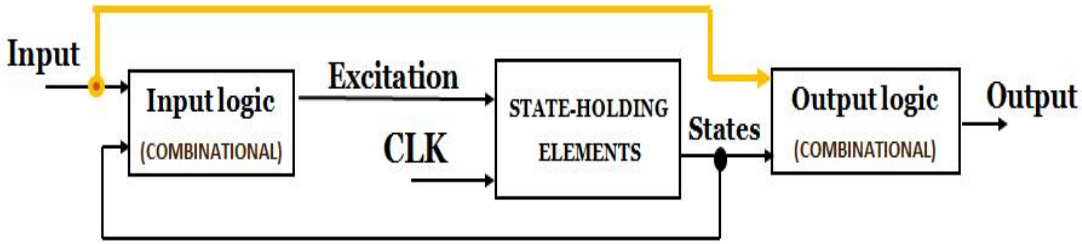
Analysis Steps:

1. Excitation logic equations
2. State transition equations
3. Output logic equations
4. State transition/output table
5. State transition graph, waveform, etc.

Design Steps:

1. State transition graph
2. State transition/output table
3. State minimization & coding
4. Chose Storage devices (DFF)
5. Excitation equations
6. Output equations
7. Logic circuit diagram





FSM Analysis

1. Excitation equations from input logic

$D0 = Q0 \cdot EN' + Q0' \cdot EN$

$D1 = Q1 \cdot EN' + Q1' \cdot Q0 \cdot EN + Q1 \cdot Q0' \cdot EN$

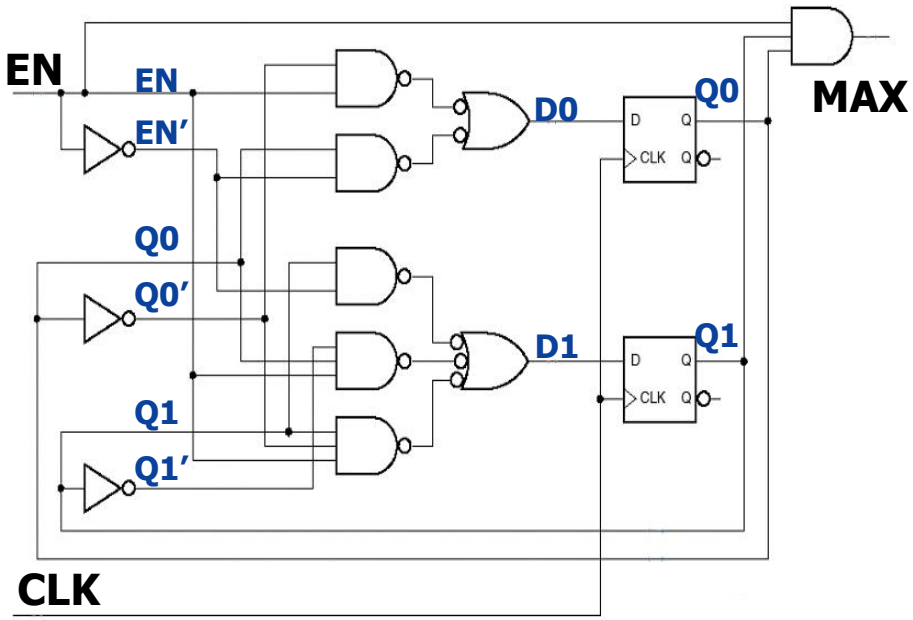
2. Output equations

$MAX = Q1 \cdot Q0 \cdot EN$

3. State transition equations

$Q0+ = D0 = Q0 \cdot EN' + Q0' \cdot EN$

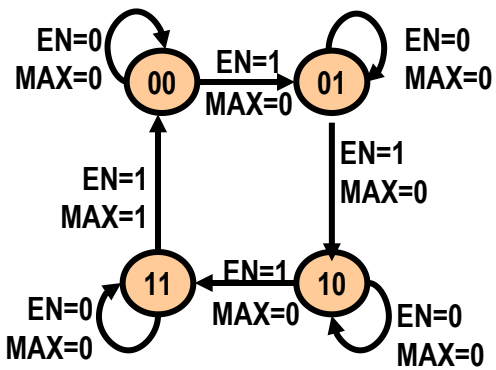
$Q1+ = D1 = Q1 \cdot EN' + Q1' \cdot Q0 \cdot EN + Q1 \cdot Q0' \cdot EN$



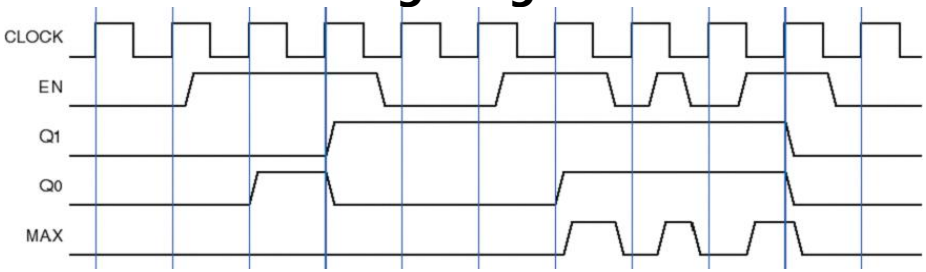
4. State transition /output table

S	EN 0	EN 1
0 0	00,0	01, 0
0 1	01,0	10, 0
1 0	10, 0	11, 0
1 1	11, 0	00, 1
Q1Q0	Q1+Q0+, MAX	

5. STG



6. Timing Diagram



2-bit Add-up Counter with Enable

Chap6.3

FSM Design

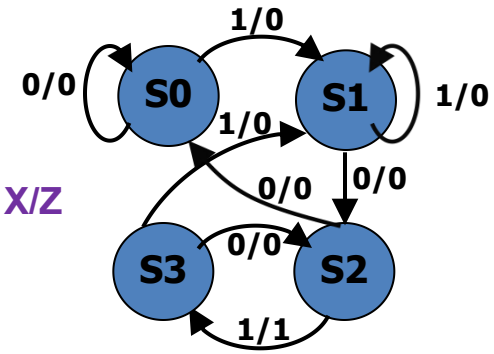
Design a Serial Data Detector of 101 (Mealy machine)

Chap6.4, 6.5

Input X=0101101001101011110……

Output Z=0001001000001010 00……

1. Mealy STG



2. State /output table

S	X=0	X=1
S0	S0 ,0	S1, 0
S1	S2 ,0	S1, 0
S2	S0, 0	S3, 1
S3	S2, 0	S1, 0
	S+, Z	

3. State minimization & coding

S	X=0	X=1
S0	S0 ,0	S1, 0
S1	S2 ,0	S1, 0
S2	S0, 0	S1 , 1
	S+, Z	

S		X=0	X=1
S0	0 0	00,0	01, 0
S1	0 1	10,0	01, 0
S2	1 0	00, 0	01, 1
Q2Q1		Q2+Q1+, Z	

4. Excitation equations

X \ Q2Q1 (Q2+)	00	01	11	10
0	0	1	X	0
1	0	0	X	0

X \ Q2Q1 (Q1+)	00	01	11	10
0	0	0	X	0
1	1	1	X	1

5. Output equations

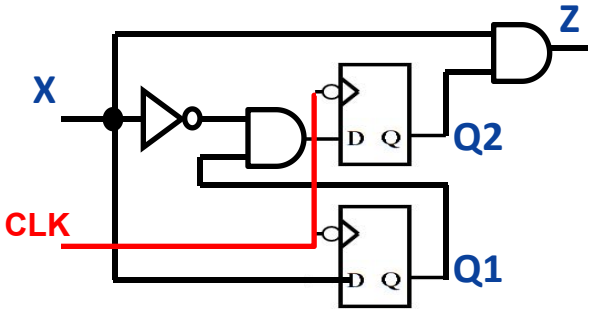
X \ Q2Q1 (Z)	00	01	11	10
0	0	0	X	0
1	0	0	X	1

Q2+=X'Q1

Q1+=X

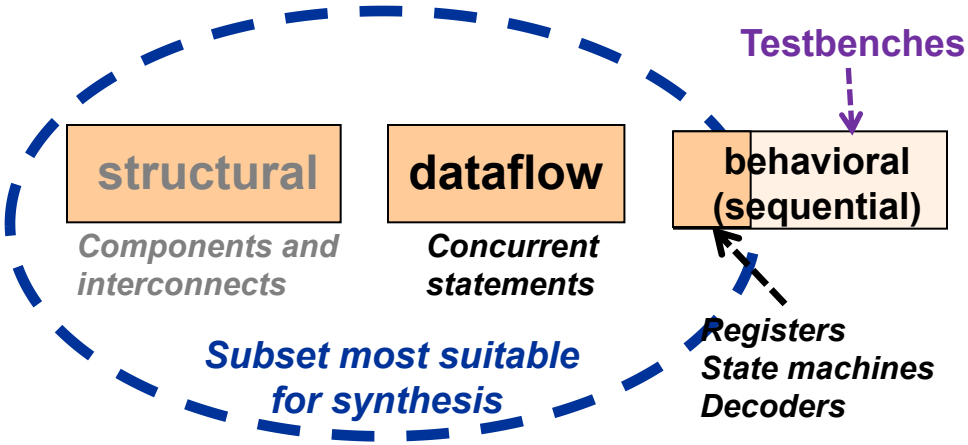
Z=Q2X

6.Logic circuit diagram (DFF)

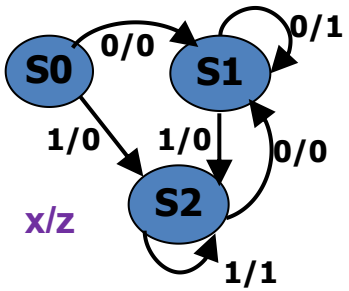


Verilog Design Styles: Sequential circuit/FSM

Chap6.4, 6.5



state		x = 0	x = 1
S0	0 0	01,0	10, 0
S1	0 1	01,1	10, 0
S2	1 0	01, 0	10, 1
	Y1Y0	Y1+Y0+, z	




```
module behavioral
  #( parameter S0=0,S1=1,S2=2 )
  (output reg z, input x, clk);
   reg [1:0] state, next_state;
     always @ ( negedge clk )
        begin
          Update State (Sequential circuit)
        end
    always @ ( state or x )
       begin
          Next state &Output (Combinational )circuit
       end
endmodule
```

```
state <= next_state;
```

```
case （state）
   S0 :    if (x==0) begin                    --PRESENT STATE/INPUT
                 next_state=S1; z=0; --NEXT STATE/OUTPUT
                end
           else      begin
                 next_state=S2; z=0;
                end
   S1:     if (x==0) begin
                 next_state=S1; z=1;
                end
           else      begin
                 next_state=S2; z=0;
                end
   S2:     if (x==0) begin
                 next_state=S1; z=0;
                end
           else      begin
                 next_state=S2; z=1;
                end
   default:          begin
                 next_state=S0; z=0;
                end
end case;
```

Examples for Sequential Digital Circuit Design

- **Flip-flop** Chap5.9, 5.10
- **Counter/Frequency Divider**
- **Serial Signal Generator**

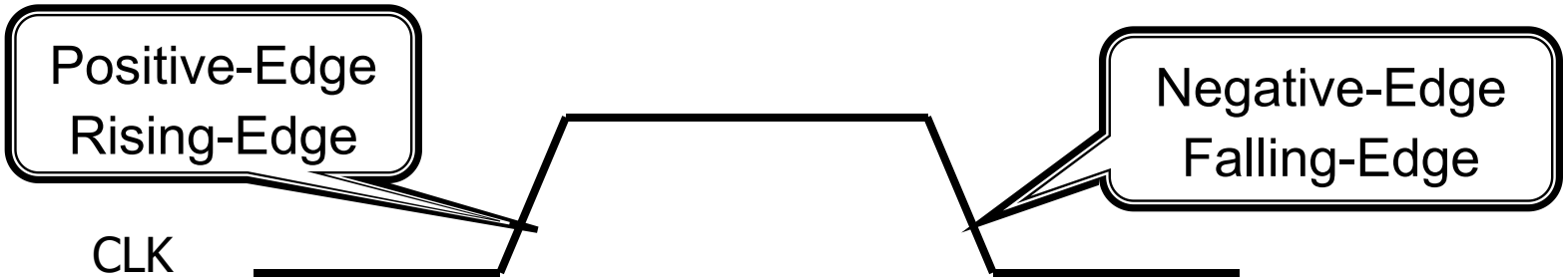
Flip-Flop

- Change its outputs only at the Rising or Falling Edge of a controlling CLK signal.
- Classified by Function

  DFF、S-R FF、J-K FF、T FF
- Classified by Structure

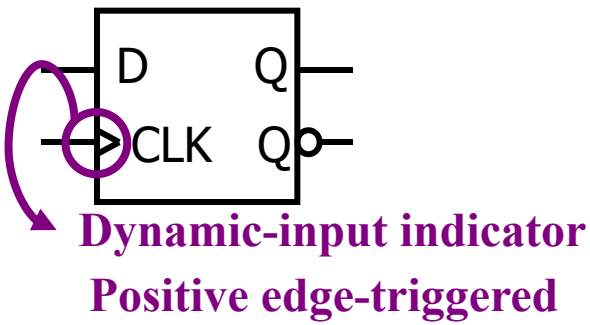
  Master/Slave、 Edge-Triggered
- Other Types Flip-Flop

  Flip-Flop with enable、Scan Flip-Flop…

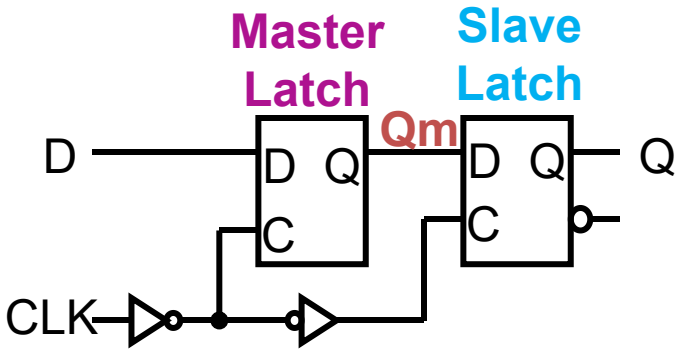


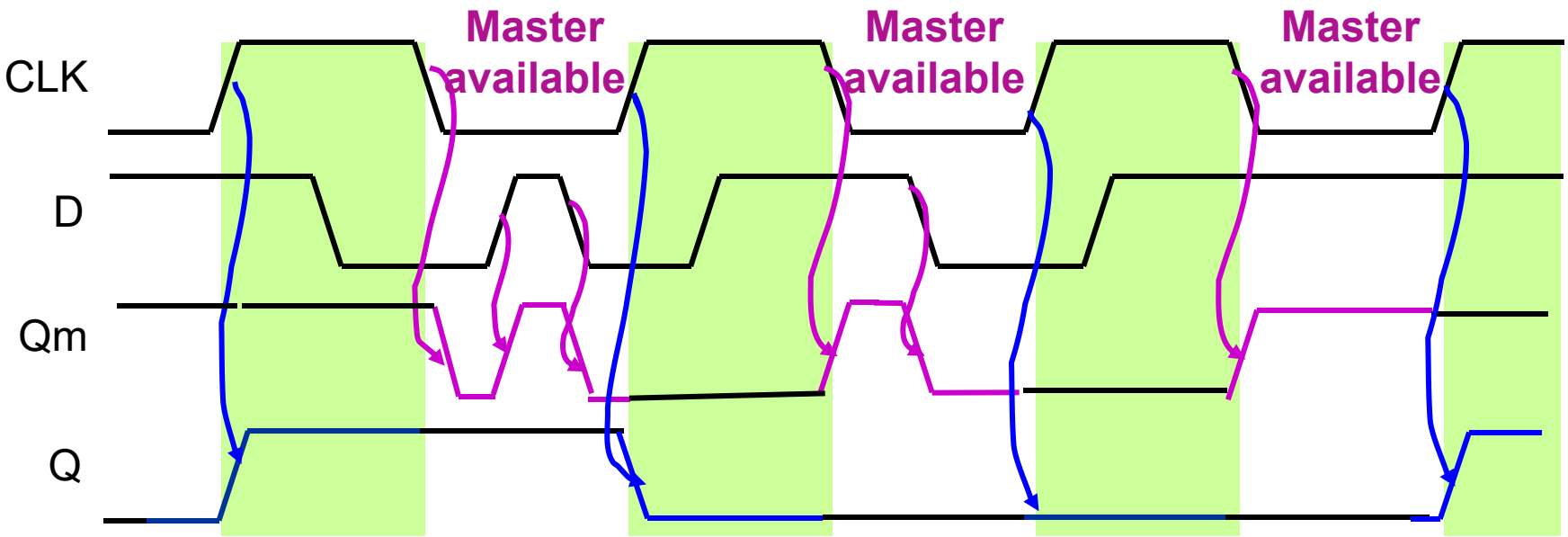
Edge-Triggered，Master-Slave DFF

Chap5.9



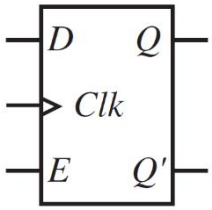
D	CLK	Q	QN
0	↑	0	1
1	↑	1	0
X	0	Hold	
X	1	Hold	



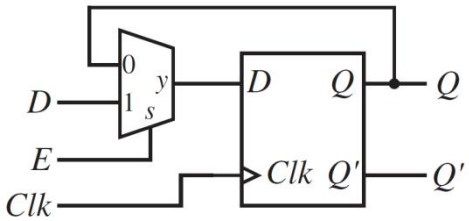


D-type Flip-Flops with other inputs

Logic Symbol



Circuit



Function table

Clk	E	D	Q	Q_{next}
0	×	×	0	0
0	×	×	1	1
1	×	×	0	0
1	×	×	1	1
↑	0	×	0	0
↑	0	×	1	1
↑	1	0	×	0
↑	1	1	×	1

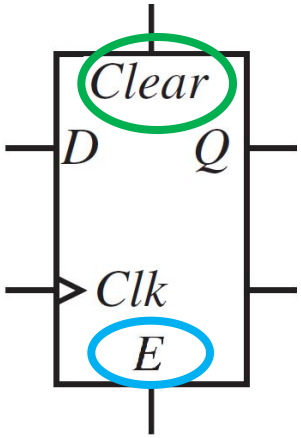
Verilog code

```
module DFF1 (
    input        CLK, D, E,  Clear,
    output  reg  Q          );

    always@ (posedge CLK  or posedge Clear)
        begin
            if  (  Clear  )
                    Q <= 0 ;
            else  if (  E  )
                    Q <= D ;
        end
endmodule
```

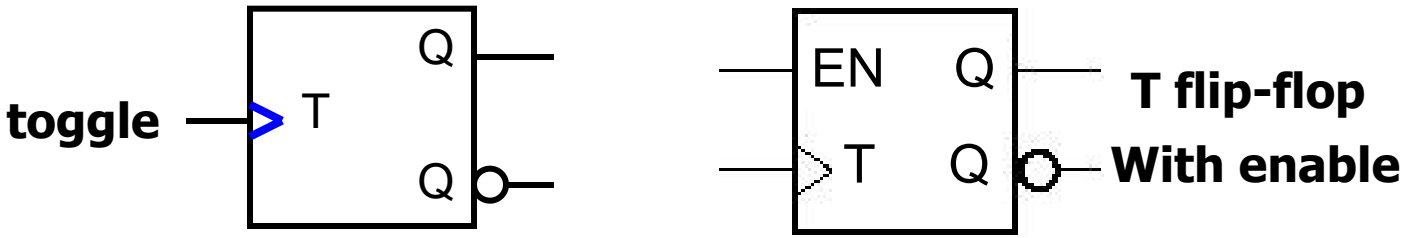
A D flip-flop with asynchronous Clear & synchronous Enable

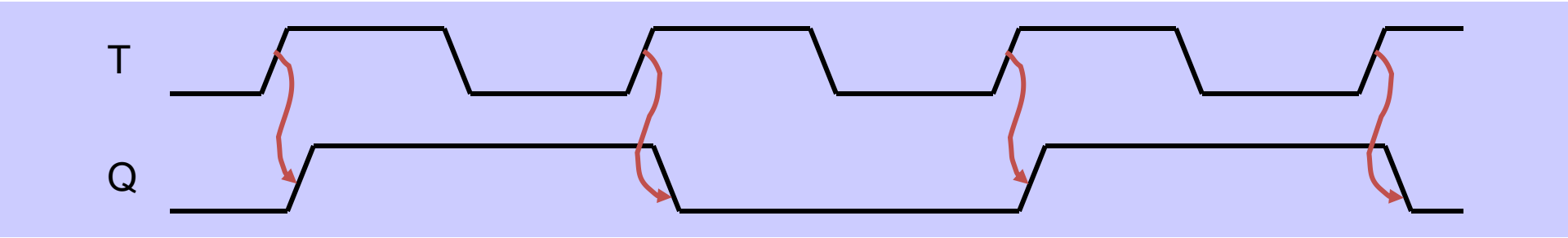
Logic Symbol

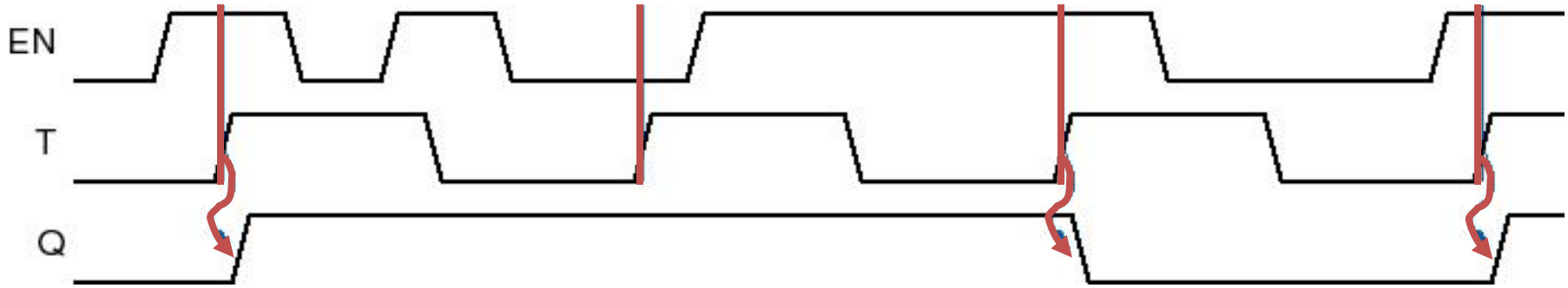


T Flip-Flop

--- changes state on every tick of the clock(T).







Exercise1

Please use a DFF to Design a T Flip-Flop.

Examples for Sequential Digital Circuit Design

- **Other Flip-flop**

Chap5.9, 5.16, 6.6.2

- **Counter/Frequency Divider**

- **Serial Signal Generator**

Counter/Frequency Divider

Counter: Chap5.9，5.16

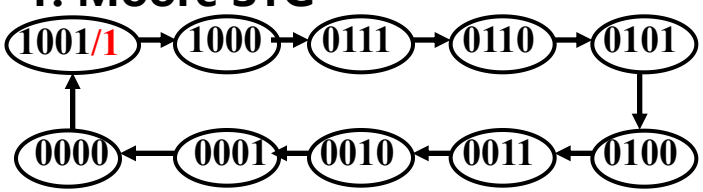
- can incorporate an incrementer or decrementer, counting the input signal upward or downward.
- General state diagram of counter contains only one single cycle.

A counter with *m* states is called a *modulo-m counter* or *divide-by-m counter*.

BCD counter

Please design a decimal downward counter.

1. Moore STG



2. State/ output table

S		S+ /Z	
S0	0000	1001/1	S9/1
S9	1001	1000/0	S8/0
S8	1000	0111/0	S7/0
S7	0111	0110/0	S6/0
S6	0110	0101/0	S5/0
S5	0101	0100/0	S4/0
S4	0100	0011/0	S3/0
S3	0011	0010/0	S2/0
S2	0010	0001/0	S1/0
S1	0001	0000/0	S0/0
Q3Q2Q1Q0		Q3+Q2+Q1+Q0+ / Z	

3. Output equations **4. Excitation equations (DFF)**

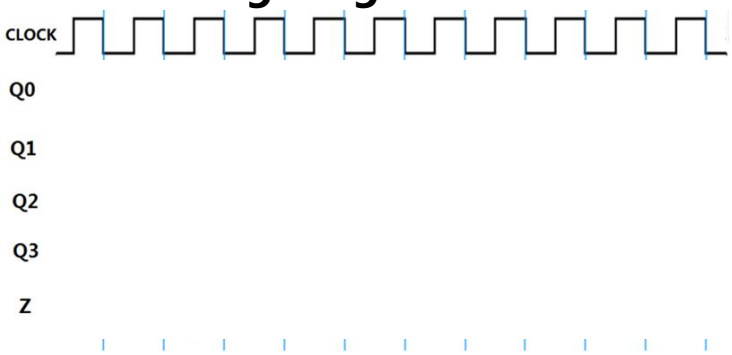
Q3+=Q3'Q2'Q1'Q0'+Q3Q2'Q1'Q0 =D3

Q2+=Q3Q2'Q1'Q0'+Q3'Q2Q0+Q3'Q2Q1=D2

Q1+=? =D1 Q0+=? =D0

5.Logic circuit diagram

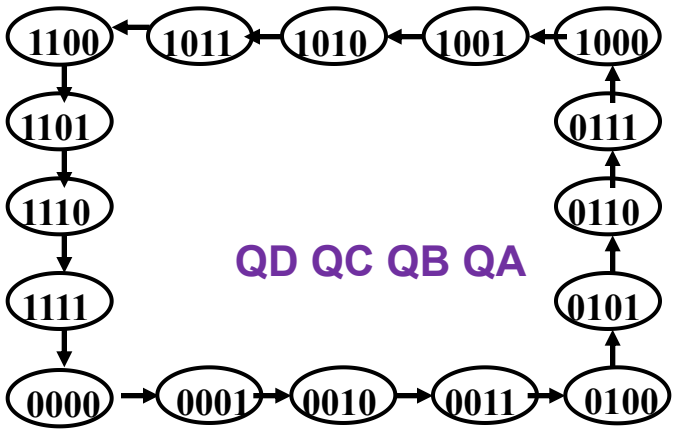
6. Timing diagram

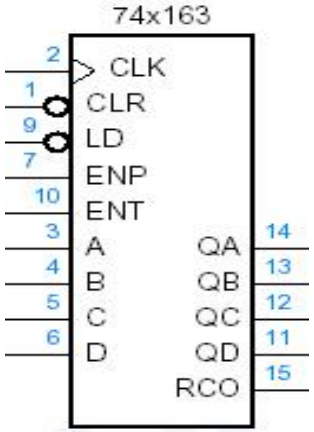


Frequency	Duty Cycle
Divided-by-2	1:2
Divided-by-4	2:4 (1:2)
Divided-by-10	4:10
Divided-by-10	2:10
Divided-by-10	1:10

Exercise 2

Please design a modulo-11 counter by 74x163.





Verilog code for 4-bit modulo-16 Counter

```
module Counter4 (reset, clk, count);
        input          reset, clk;
        output [3:0]   count;
        reg    [3:0]   count;

        always @( negedge clk )
          begin
            if (reset) count <= 4'b00_00;
            else       count <= count + 1;
          end
endmodule
```

Constant Definition for initializing counter

When *count* reaches 1111, the next count taken is 0000

Examples for Sequential Digital Circuit Design

- Other Flip-flop
- Counter/Frequency Divider
- Serial Signal Generator

Exercise 3

Please design a serial signal generator which output is ‘110100’.

Verilog code for serial signal generator

```
module  fsm1_seq101
   #( parameter   S0=0,S1=1,S2=2,S3=3,S4=4,S5=5 )
 ( input      clk,
   output   reg  Y      );
   reg   [2:0]  state, next_state;

    always @  ( posedge clk )
    begin
       Update State (Sequential circuit)

    end

    always @  (   state   )
    begin
       Next State& Output (Combinational circuit)

    end
endmodule
```

```
state <= next_state;
```

```
case （state）
   S0 : begin
            next_state=S1;  Y=1;
        end
   S1:   begin
            next_state=S2;  Y=1;
         end
   S2:   begin
            next_state=S3;  Y=0;
         end
   S3:   begin
            next_state=S4;  Y=1;
         end
   S4:   begin
            next_state=S5;  Y=0;
         end
   S5:   begin
            next_state=S0;  Y=0;
         end
   default:  begin
            next_state=S0;  Y=1;
         end
end case
```

Q1: Is it a Moore or Mealy machine?

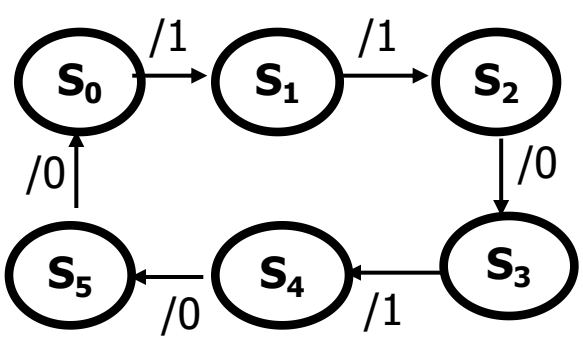
Q2: What's the meaning of default branch?

State Assignment (Coding)

Chap6.5.2

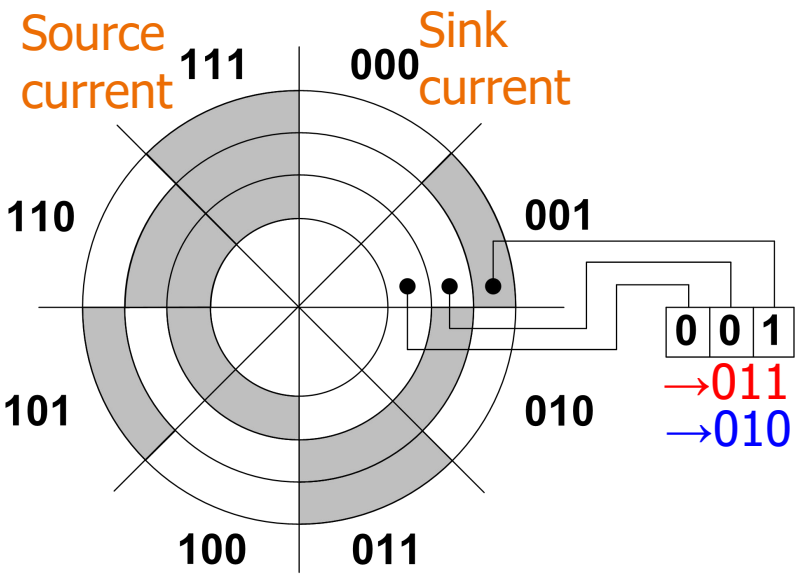
State assignment: assigning a specific combination to each named state.

- ✓ Choose an initial, simplest coded state .
- ✓ Are there unused states? Don't limit the choice of coded states to the first s n-bit integers.
  - ☐ Decompose the set of state variables into individual bits of fields, where each bit or field has a well-defined meaning with respect to the input effects or output behavior of the machine.
  - ☐ Minimize the number of state variables that change on each transition.

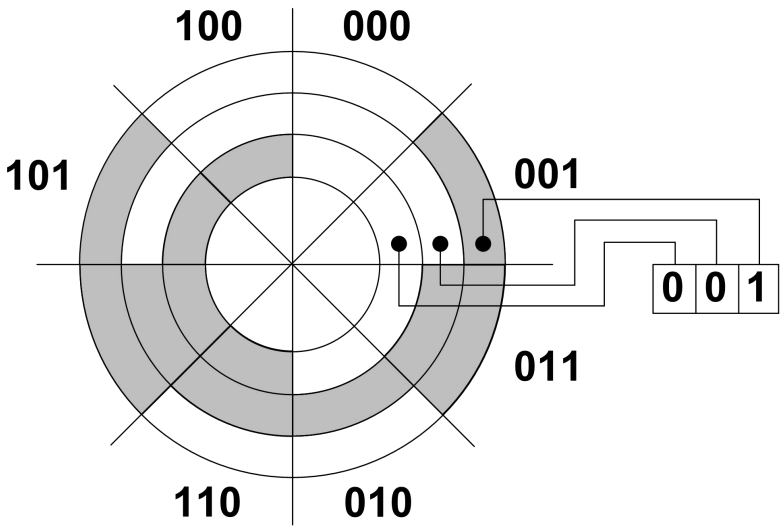


State	Simplest	Decomposed	One-Hot	Almost one-hot
S0	000	001	000001	00000
S1	001	101	000010	00001
S2	010	010	000100	00010
S3	011	110	001000	00100
S4	100	011	010000	01000
S5	101	111	100000	10000

Gray code



Binary code



Gray code

- Some devices in mechanical systems indicate positions by closing and opening switches. The inter-mediate states maybe cause logic confusion and raise circuits power consumption.
- Gray code is a binary numeral system where two successive code words differ in only one binary bit. So it can be used to avoid inter-mediate states, greatly reducing logical confusion from one state to the next, reducing power consume in circuit state changes.

From Binary code to Gray code

Gray-code	Decimal	Binary
0 0 0	0	000
0 0 1	1	001
0 1 1	2	010
0 1 0	3	011
1 1 0	4	100
1 1 1	5	101
1 0 1	6	110
1 0 0	7	111

Reflecting

from 1-bit to n-bit

Binary: 0 1 0 0 1 1 1 0 1

Gray-code: 1 1 0 1 0 0 1 1

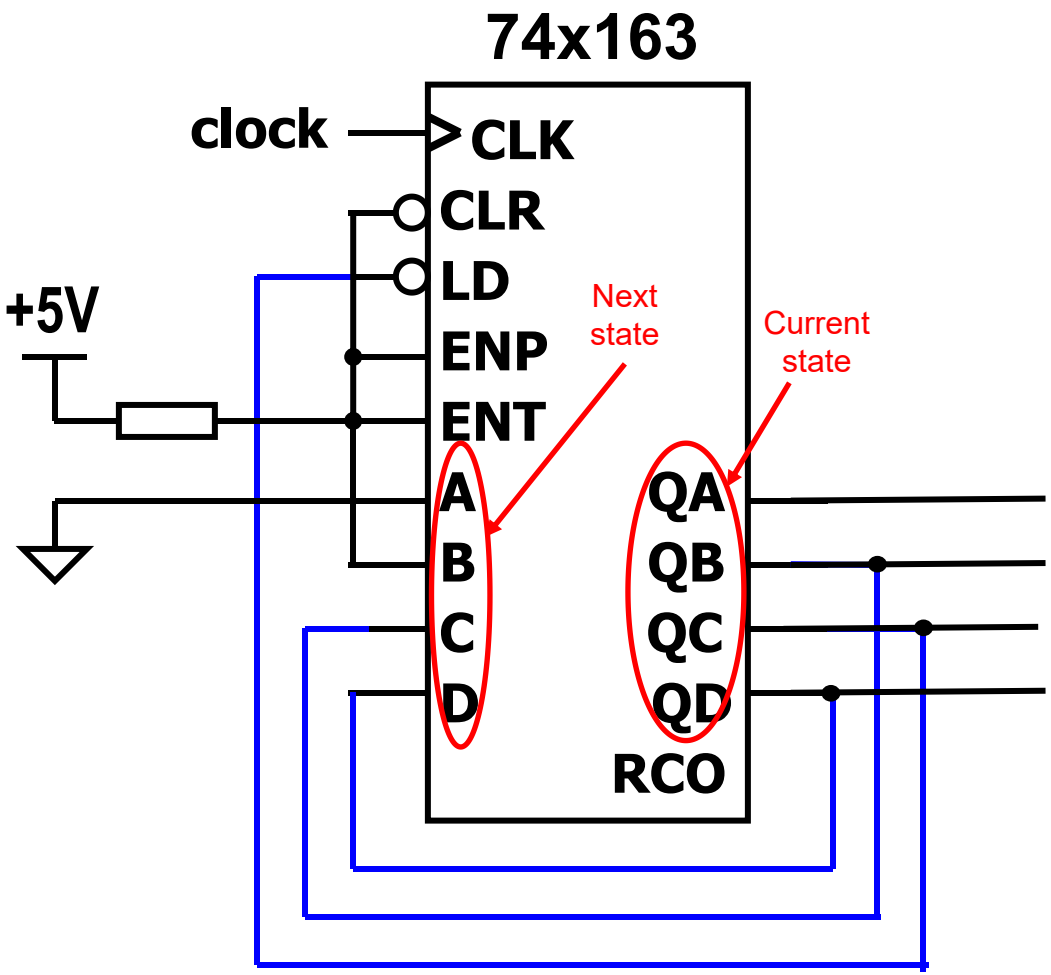
– **XOR**

from right to left

BCD code

Decimal	BCD Code	2421 Code	Excess-3	Bi-quinary	1-out-of-10
0	0000	0000	0011	0100001	1000000000
1	0001	0001	0100	0100010	0100000000
2	0010	0010	0101	0100100	0010000000
3	0011	0011	0110	0101000	0001000000
4	0100	0100	0111	0110000	0000100000
5	0101	1011	1000	1000001	0000010000
6	0110	1100	1001	1000010	0000001000
7	0111	1101	1010	1000100	0000000100
8	1000	1110	1011	1001000	0000000010
9	1001	1111	1100	1010000	0000000001
		unused			
	1010	0101	0000	0000000	0000000000
	1011	0110	0001	0000001	0000000011
	1100	0111	0010	0000010	0000000101
	1101	1000	1101	0000011	0000000110
	1110	1001	1110	0000101	0000000111
	1111	1010	1111	...	...

Unused state & Self-correcting



A modulo-8 counter

QD/D	QC/LD	QB/C	B	A
0	0	0	1	0
0	0	1	1	0
0	1	0	x	x
0	1	1	x	x
1	0	0	1	0
1	0	1	1	0
1	1	0	x	x
1	1	1	x	x

0001 0011 0100->0101

0000-> 0010-> 0110-> 0111->

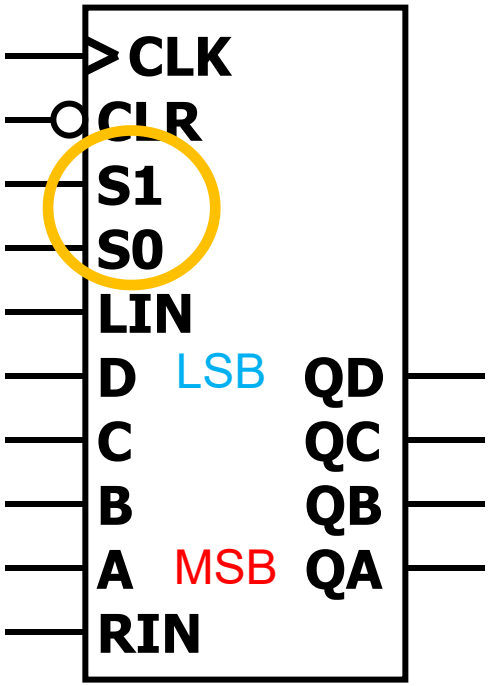
$Q_DQ_CQ_BQ_A$

<-1111<- 1110<- 1010<- 1000<-

1100->1101 1011 1001

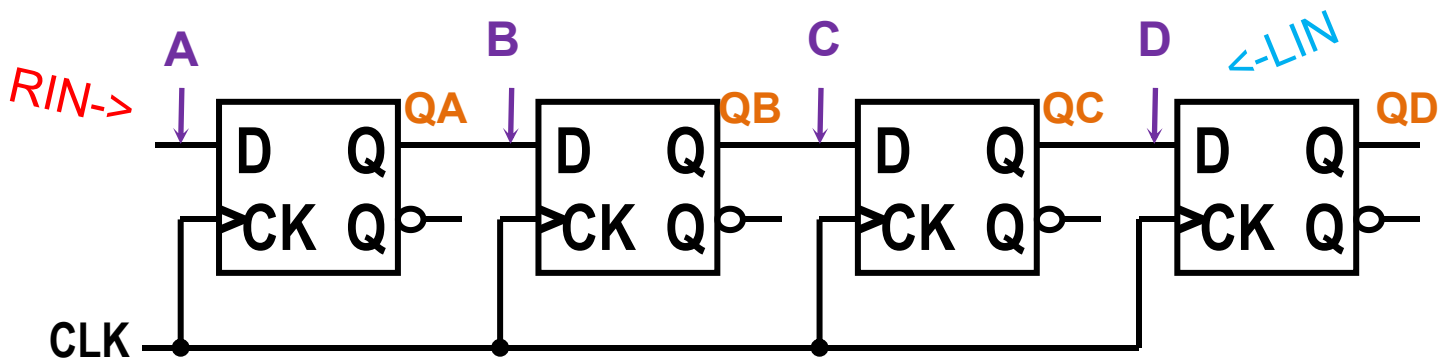
Unused state & Self-correcting:

Other Counters based on a Shift Register



4-bit universal shift register

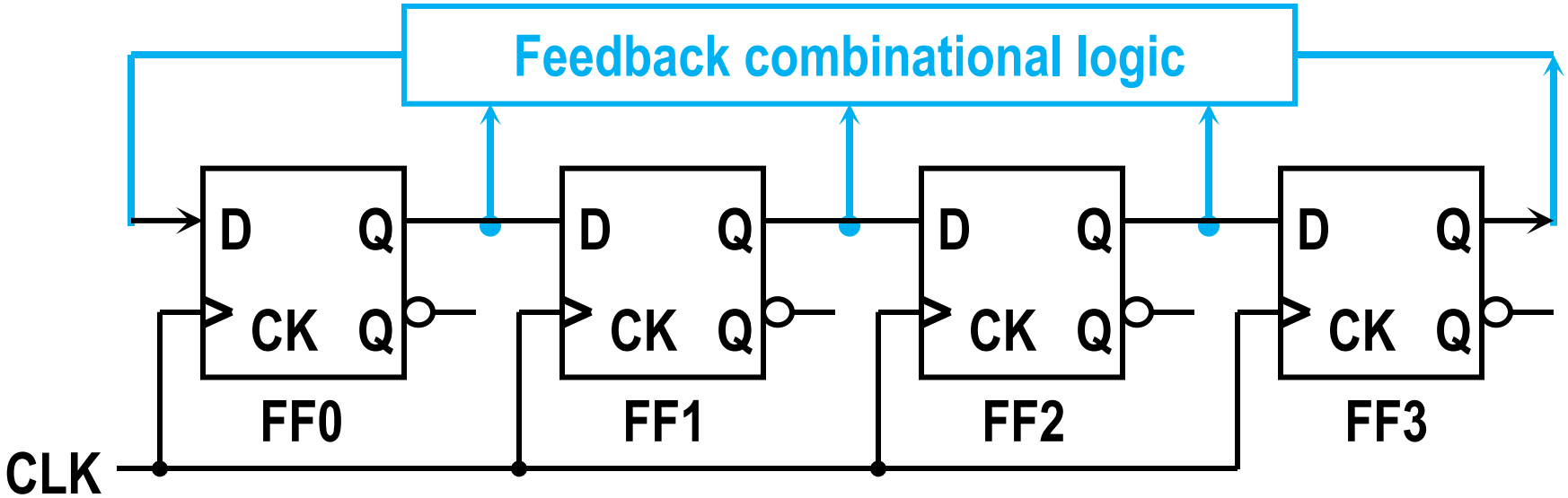
Function	Inputs S1	Inputs S0	Next state QA*	QB*	QC*	QD*
Hold	0	0	QA	QB	QC	QD
Shift right	0	1	RIN	QA	QB	QC
Shift left	1	0	QB	QC	QD	LIN
Load	1	1	A	B	C	D



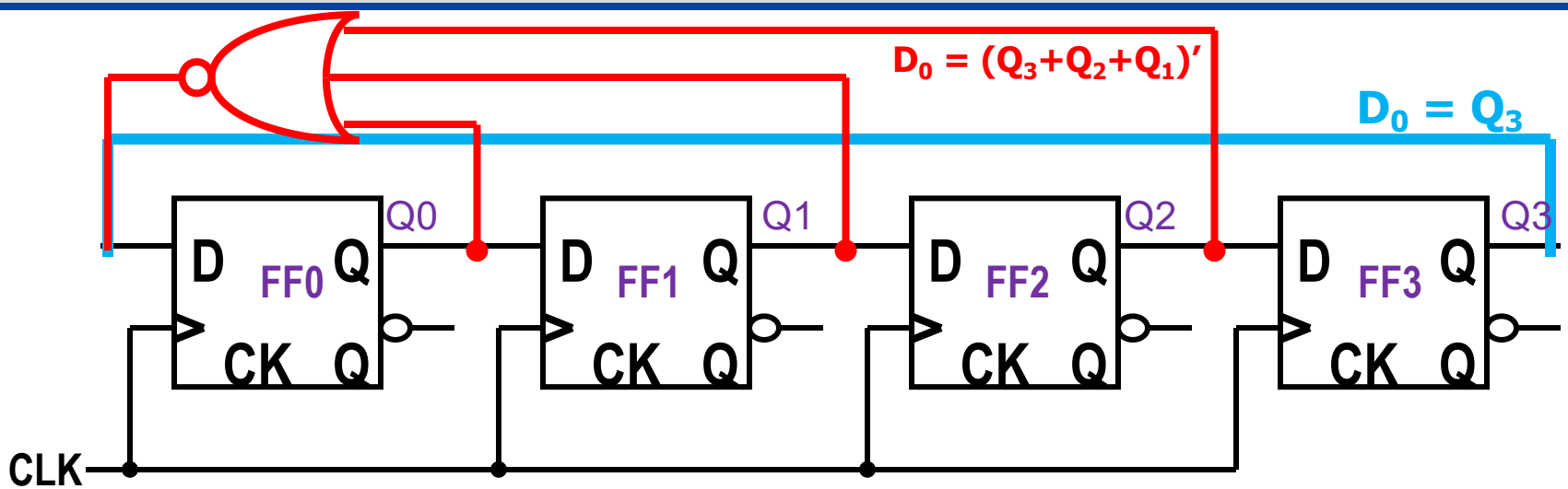
Shift Register Counter

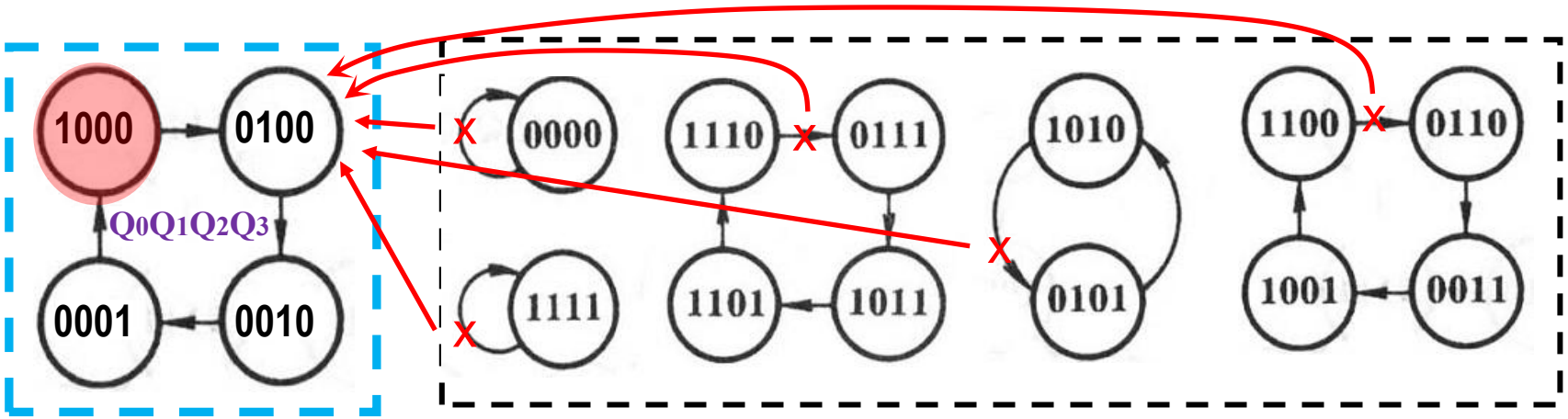
Unlike a binary counter, a shift-register counter **does not count** in an ascending or descending binary sequence. It is useful in many "control" units for **holding several status** by defining **the serial input** D_0.

$$D_0 = F(Q_0, Q_1, \dots, Q_{n-1})$$



n-bit Ring Counter



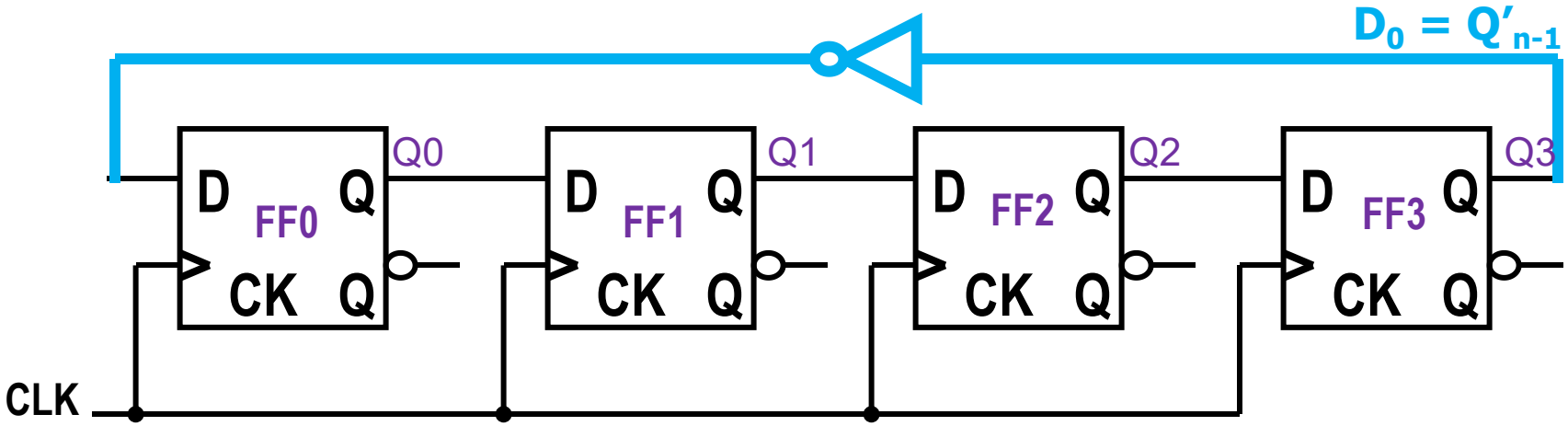


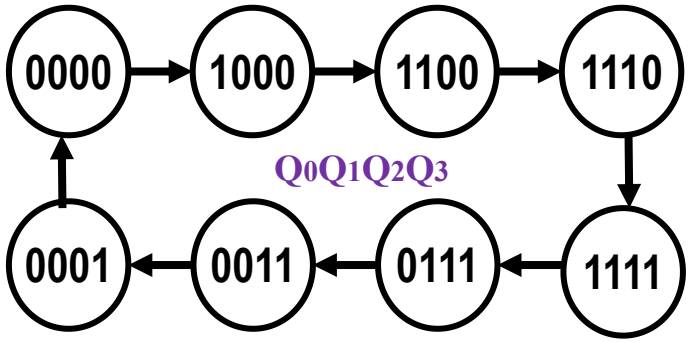
Used Status (on-hot)

Unused Status(2^n-n)

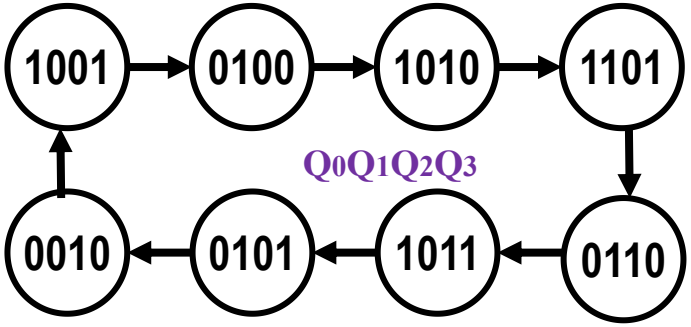
self-correcting (else branch)

n-bit Johnson(Wisted-ring) Counter





Used Status



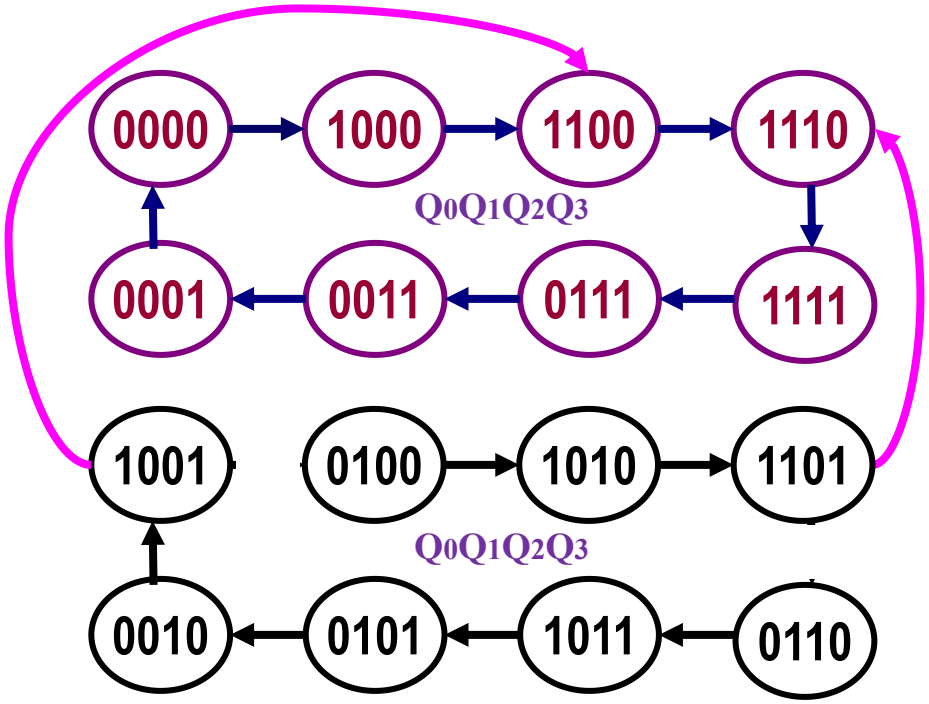
Unused Status(2^n-$2n$)

self-correcting (else branch) design: STG changing, Watchdog, Preset

Self-correcting Design for Johnson Counter

D0 Q2Q3 \ Q0Q1	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	1	1	1	1

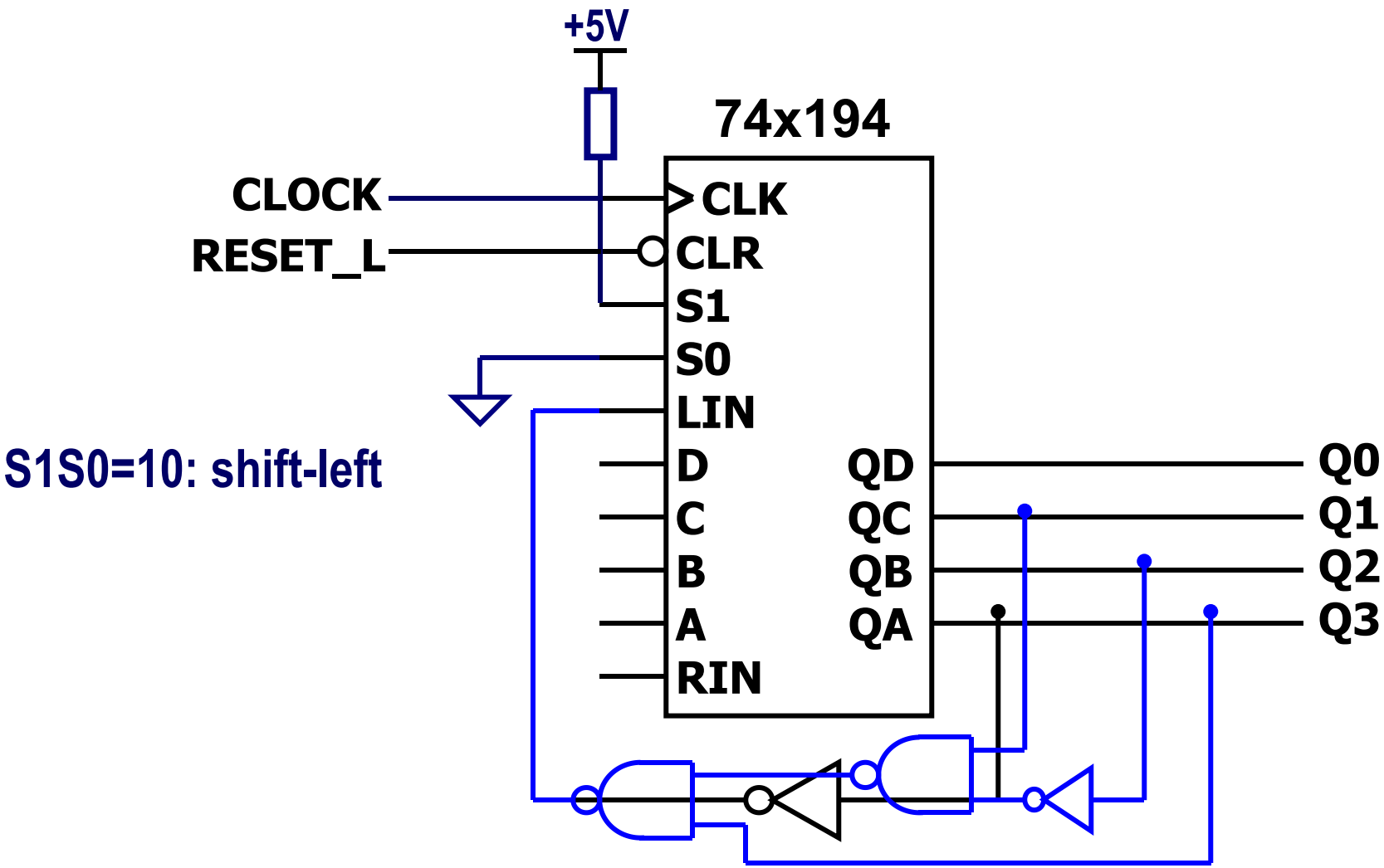
D0 Q2Q3 \ Q0Q1	00	01	11	10
00	1	1	1	1
01	0	0	1	1
11	0	0	0	0
10	1	1	1	1



D0 = Q3′ + Q2′·Q0

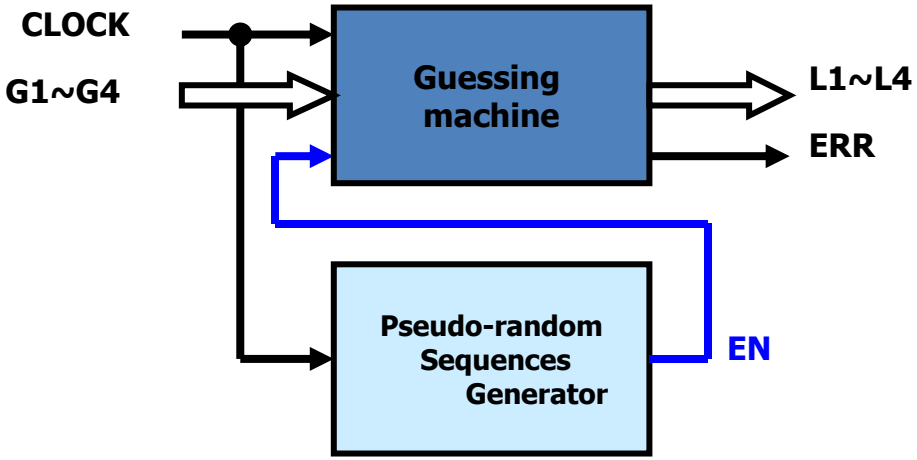
最小成本

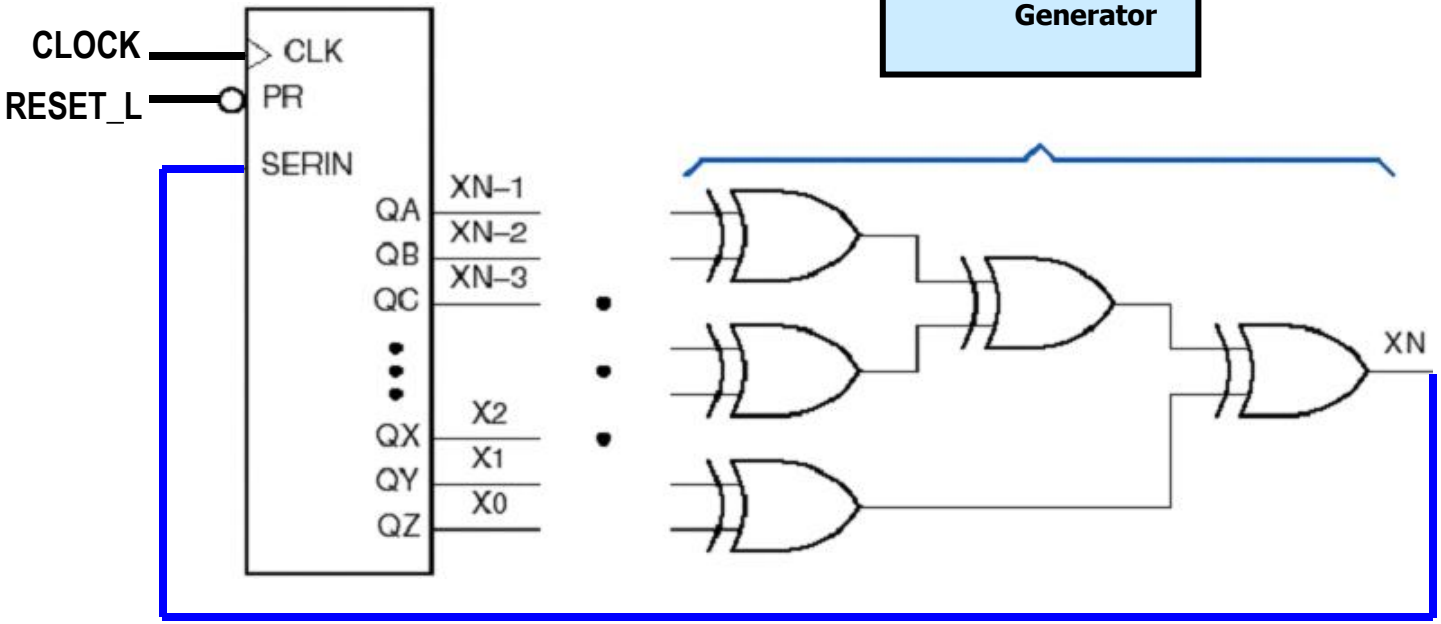
Self-correcting Design for Johnson Counter



Linear Feedback Shift Register Counters

LFSR counter is *a maximum-length sequence generator* having 2^n-1 valid states





Digital Logic Design and Application

Topic 2 - Principles for digital design

Yan Bo

School of Information & Communication Engineering, UESTC

Email: yanboyu@uestc.edu.cn

THE END OF TOPIC2-2

Required reading

- **Digital System Design with Verilog &VHDL**

  2nd Edition，Enoch O.Hwang著，阎波 朱晓章 姚毅改编

  - ✓ *Chapter 5, Sequential Circuit*
  - ✓ *Chapter 6, Finite-State Machines*

- **Digital Design: Principles and Practices**

  5th Edition, John F. Wakerly

  - ✓ *Chapter 7, Sequential Logic Design Principles*
  - ✓ *Chapter 8, Sequential Logic Design Practices*

- **数字逻辑设计及应用**

  姜书艳等，电子科技大学出版社

  - ✓ *Chapter 6 , Memory Circuits*
  - ✓ *Chapter 7, Sequential Logic Circuit Design*

- **数字集成电路与系统设计**

  李广军、郭志勇等，电子工业出版社

  - ✓ *Chapter 4 , Verilog HDL基础*
  - ✓ *Chapter 5, 数字逻辑电路的Verilog RTL建模与设计*

Homework3

P211　5.1、5.3、 5.4、 5.8、5.10

P271　6.1（a）、6.3（a）、6.4、6.7